

Not Only Rewards but Also Constraints: Applications on Legged Robot Locomotion

Yunho Kim , Hyunsik Oh , *Graduate Student Member, IEEE*, Jeonghyun Lee , *Graduate Student Member, IEEE*, Jinhyeok Choi , Gwanghyeon Ji , Moonkyu Jung , Donghoon Youm , *Graduate Student Member, IEEE*, and Jemin Hwangbo , *Member, IEEE*

Abstract—Several earlier studies have shown impressive control performance in complex robotic systems by designing the controller using a neural network and training it with model-free reinforcement learning. However, these outstanding controllers with natural motion style and high task performance are developed through extensive reward engineering, which is a highly laborious and time-consuming process of designing numerous reward terms and determining suitable reward coefficients. In this article, we propose a novel reinforcement learning framework for training neural network controllers for complex robotic systems consisting of both *rewards* and *constraints*. To let the engineers appropriately reflect their intent to constraints and handle them with minimal computation overhead, two constraint types and an efficient policy optimization algorithm are suggested. The learning framework is applied to train locomotion controllers for several legged robots with different morphology and physical attributes to traverse challenging terrains. Extensive simulation and real-world experiments demonstrate that performant controllers can be trained with significantly less reward engineering, by tuning only a single reward coefficient. Furthermore, a more straightforward and intuitive engineering process can be utilized, thanks to the interpretability and generalizability of constraints.

Index Terms—Constrained reinforcement learning (RL), legged locomotion, RL.

I. INTRODUCTION

RECENTLY, learning-based methods have gained significant popularity for designing controllers in complex robotic systems. These techniques employ a neural network as a controller, mapping the robot's observations to control inputs [1]. The network parameters are trained using either expert demonstration data (imitation learning) [2], [3], [4] or interaction data [reinforcement learning (RL)] [5], [6], [7]. Since expert demonstration data are often limited for robotic systems compared with readily available interaction data from both

physics simulations and real-world experiments, RL has become a dominant approach. These methods offer distinct advantages over model-based approaches, especially in scenarios where the system's complexity increases with more available contacts (e.g., legged robots, high-degree-of-freedom robot hands) and environmental uncertainties grow due to factors, such as sensor noise and disturbances.

When engineers build a neural network controller with RL, they typically do the following steps: First, design a neural network architecture with inductive bias based on the observation and action spaces and second, generate abundant environment interaction scenarios (e.g., random terrains for legged robot locomotion and diverse object meshes for robot hand grasping); and third, design reward terms and tune their reward coefficients until a high-performing controller is achieved, which satisfies the robot's physical constraints (e.g., joint torque and velocity limits) and exhibits a natural motion style. Among the three steps, the last one is the most time-consuming because the tuning process has to be repeated several times. Finding the relative weights for each of the reward terms based on the resulting robot's motion is not trivial because there are often more than ten reward terms [8], [9].

In this work, we want to raise the following fundamental question: *Why constraints have not been used explicitly to train policies for complex robotic systems?* RL can be thought of as a numerical optimization problem: Finding the neural network parameters that maximize the objective, which is the weighted sum of the reward terms. As the system becomes more complex and the desired motions become more agile, it is inevitable to put more effort into reward engineering to make the objective surface smooth with fewer local optima and guide the optimization toward the desired point. However, in numerical optimization problems, constraints are also utilized to narrow down the solution search space to the region that is desirable for the engineer or feasible in the actual system.

Similarly, if constraints can be explicitly defined in the RL framework, instead of relying solely on rewards, there can be several advantages. First, the training pipeline will be more generalizable across similar robot platforms. If the learning framework is only designed with rewards, the performance of the controller can vary across multiple robots due to the differences in feedback signals for each reward component. These variations result in different control behaviors, making the controller's performance unique to each robot's specific characteristics and

Manuscript received 14 December 2023; revised 10 March 2024 and 25 April 2024; accepted 5 May 2024. Date of publication 14 May 2024; date of current version 30 May 2024. This paper was recommended for publication by Associate Editor C. Della Santina and Editor S. Behnke upon evaluation of the reviewers' comments. This work was supported by Samsung Research Funding & Incubation Center for Future Technology at Samsung Electronics under Grant SRFC-IT2002-02. (Corresponding author: Jemin Hwangbo.)

The authors are with Robotics and Artificial Intelligence Lab, KAIST, Daejeon 34141, South Korea (e-mail: jhwangbo@kaist.ac.kr).

Data is available online at <https://youtu.be/KAlm3yskhvM>.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TRO.2024.3400935>, provided by the authors.

Digital Object Identifier 10.1109/TRO.2024.3400935

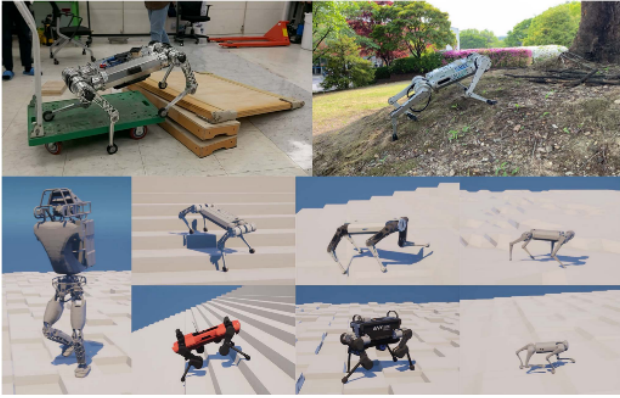


Fig. 1. Several legged robots trained with our learning framework to locomote in challenging terrains. The robots include six quadrupedal robots and one bipedal robot.

dynamics. On the other hand, constraints can be used as generalizable conditions that the controller should satisfy even when the robot changes. For instance, when we include a trotting gait pattern constraint in the training of locomotion controllers, it can be applied consistently to all quadrupedal robots, irrespective of their size, mass, and morphology. Second, the engineering process will be more straightforward and less time-consuming. There will be fewer reward terms to adjust, as the rewards that previously acted as soft penalties to accommodate the robot's hardware limits or define its motion style are no longer needed. Instead, they are defined as constraints to drive the policy toward the approximate desired region that coincides with the engineer's intent. Furthermore, constraint parameters and limits can be set more intuitively, compared with reward coefficients that correspond to the relative importance of each term, because they have physical meanings (e.g., joint angle limits) and can even be set automatically from the robot description files (e.g., URDF files).

To this end, we propose an RL framework for complex articulated systems consisting of both rewards and constraints. Appropriate constraint types are suggested, where each can be utilized in a way that suits the engineer's intent. Inspired by previous works in the constrained RL literature (also known as safe RL), an efficient policy optimization algorithm is suggested to search for a policy that maximizes the reward while satisfying multiple constraints with a negligible amount of additional computation cost. The proposed learning framework is used to train controllers for legged robot locomotion in challenging terrains, which was possible only with considerable reward engineering. Through extensive experiments conducted in both simulation and the real world, involving multiple robots with different morphologies (e.g., leg numbers, mechanical leg designs) and properties (e.g., robot mass, actuator parameters), as shown in Fig. 1, we demonstrate the effectiveness of our framework. Specifically, we show that high-performance controllers can be obtained with significantly less reward engineering (i.e., only three reward coefficients and a single reward coefficient modification) and by exploiting more generalizable and straightforward constraints.

Unlike other related works [10], [11] that focused on safety for the benefit of constraints, our work investigates a new advantage of utilizing constraints from a practical engineering perspective. To the best of the authors' knowledge, our work is the first to show highly robust simulation and real-world robot performance for multiple legged robots (i.e., six quadrupedal robots and one bipedal robot) using constrained RL and demonstrate the benefits of constraints from an engineering standpoint.

A. Contributions

The primary contribution of this article is to provide new perspectives on designing a learning-based controller for complex articulated systems. Our experiments demonstrate how using constraints makes the engineering process more efficient, compared with relying solely on well-tuned rewards to reflect the engineer's intent. This improvement stems from the generalizability and interpretability of constraints, along with the reduced need for extensive reward designs. The concrete contributions are elaborated on below.

- 1) We introduce an RL framework for complex articulated systems consisting of both *rewards* and *constraints*, drawing inspiration from existing works in constrained RL. Our approach incorporates suitable constraint types and an efficient policy optimization algorithm. The framework is designed to be highly scalable in terms of the number of constraints while enabling high-performance policy learning.
- 2) Through extensive simulation and real-world experiments involving multiple legged robots with varying morphologies and physical properties (i.e., six quadrupedal robots and one bipedal robot), we demonstrate the capability of leveraging constraints in the learning pipeline to improve robot generalizability and engineering intuitiveness, which are difficult to achieve when relying solely on rewards, while maintaining highly robust control performance.
- 3) Building on our encouraging experimental results, we present an intriguing direction for considering constraints as a new tool that can be used in conjunction with or instead of rewards to design neural network controllers. This alleviates the burden of tedious reward engineering, which is a major bottleneck for robot control with RL.

B. Outline

The rest of this article is organized as follows. After Section II reviewing relevant literature, Section III provides short background information about constrained RL. Section IV explains the proposed learning framework, including constraint types and the policy optimization algorithm. Section V demonstrates how we applied the proposed framework for legged robot locomotion. Section VI illustrates the implementation details and experimental results. Finally, Section VII concludes this article with a discussion and future research directions.

II. RELATED WORK

A. Control of Complex Robotic Systems

In model-based control literature, engineers typically do the following to design a controller for a system: First, analyze the system's kinematic and dynamic equations; second, define heuristics or objectives and constraints that represent the desired motion; and third, design a control system with numerical optimization or simpler techniques (e.g., proportional–integral–differential control) and generate control inputs. These approaches are effective when the system is relatively simple and the environment is almost deterministic. However, they become challenging when the system becomes more complex and the environmental uncertainties grow.

To handle these challenges, learning-based control methods with deep neural networks are actively applied to controlling complex articulated systems. The learning-based control literature can be subdivided into two categories based on the data used for training the neural network: Imitation learning and RL. Imitation learning trains the network to mimic the expert's behavior through expert demonstration data. The network can be trained directly via behavior cloning with data aggregations [2] or indirectly by matching the expert's state distribution [4], [12]. However, demonstration data for complex articulated systems are often limited or not available. RL is an appealing alternative in these cases. It aims to train a controller that maximizes the expected reward sum by using interaction data, rather than expert demonstration data, which can be readily obtained in both simulation and the real world. Neural networks can be used to parameterize the dynamic model of the environment (model-based RL) [13], [14] or the controller itself that maps observations to control inputs (model-free RL). In this work, we are mostly focused on model-free RL methods, which have been successfully applied to controlling various complex systems, including animated characters [15], high-DoF robot hands [7], drones [5], and mobile manipulators [16] with well-engineered dense reward signals.

B. Legged Robot Locomotion

Legged robots have a great chance of navigating a variety of rough terrains that wheeled robots find challenging. They may be deployed anywhere that people and animals can go and traverse the region by carefully selecting footholds and altering the base motions. However, the complexity of the hardware and the underactuated nature makes it difficult to design a control algorithm for these hardware platforms. Trajectory optimization techniques are a widely used methodology in the domain of model-based control. In these methods, the control is separated into two modules: Planning and tracking. The planning module generates base motion and foot trajectories using numerical optimization or heuristics to satisfy the given high-level commands from the user, such as a velocity command and a desired foot contact sequence. The tracking module then generates actuation torques to follow the plan. Although the effectiveness of these methods has been demonstrated using physical robots for both blind locomotion [17], [18] and perceptive locomotion [19],

[20], there are challenges associated with their application in wild environment settings due to the environmental uncertainties, sensor noise, and unforeseen corner cases.

Model-free deep RL has recently risen to attention as an alternative method for legged locomotion. These techniques use neural networks to model a locomotion controller, and its parameters are automatically trained to maximize the designed reward signals. Because RL requires lots of data to get superior control performance, a recent trend is to train the controller in the physics simulator [1], [6], [21], [22] and deploy it in the real world (i.e., sim-to-real). Techniques including domain randomization [23], domain adaptation, and hybrid simulation [1] are suggested to bridge the gap between the simulation and the real world. Previous research on quadruped robots demonstrated that RL controllers may be very reliable and generalize to difficult contexts by simulating a large amount of related experiences. In particular, Lee et al. [6] and Kumar et al. [24] demonstrated robust blind locomotion capabilities in a variety of hard conditions, including slippery and rocky hills, through teacher–student learning in multiple simulated rigid terrains. Choi et al. [9] demonstrated high-speed quadrupedal locomotion in a variety of deformable terrains, such as soft beach sand, using efficient granular media simulation. Miki et al. [21] and Agarwal et al. [25] introduced improved RL locomotion controllers by exploiting exteroceptive information, such as terrain geometries around each foot or front-facing depth camera data. Furthermore, RL controllers demonstrated high-speed dynamic locomotion of small-scale quadruped robots [8], [26], multiple gait transitions [27], [28], human-size bipedal robot locomotion [29], and even more agile movements, such as parkour [30] or environment interactions [31].

Although the advantages of using model-free deep RL for legged robot locomotion have been shown in various previous works in perspective of the control performance, less attention is given to how the controllers are engineered. To achieve such a robust controller with natural and smooth joint movements, a variety of reward terms (often ten or more) are designed first, and the reward coefficients for each of the terms are extensively tuned until satisfactory performance is achieved. This reward engineering process is the most time-consuming part of designing the controller (this procedure often takes several months). Finding the relative weights for each of the reward terms based on the resulting robot's motion is not trivial and it highly depends on the engineer's experience and intuition.

There are some previous works to reduce the burden of reward engineering. Escontrela et al. [32], Wu et al. [33], and Fuchioka et al. [34] utilized an auxiliary task to imitate the natural motions present in expert demonstration data, derived either from animal motions or trajectory optimization. However, the motions that the controller can generate are limited to the available expert demonstration data. Feng et al. [35] proposed a morphology randomization technique to generate a general controller for quadruped robots and remove the robot-specific engineering process. However, training a single controller for a broad range of quadruped robot morphologies results in a conservative controller and is a well-known problem of domain randomization techniques. Furthermore, their experiments were

restricted to indoor settings with simple behaviors (e.g., go forward, turn in place), limiting their ability to demonstrate high controller robustness.

In this work, we propose a learning framework that is formulated with both rewards and constraints. Because constraints are generalizable, interpretable, and can be utilized to guide the policy to the desired region, our method reduces the burden of extensive reward engineering by requiring fewer reward terms (only three) to tune and making the training process more general across similar robot platforms.

C. Constrained RL

In RL, the agent learns to maximize the expected reward sum through trial and error. In many realistic settings, however, giving an agent complete freedom may be unacceptable. For example, consider a legged robot learning to move forward with minimal torque usage. It may be undesirable for the robot to encounter body contact rather than the feet (e.g., self-collision, contact between the ground and the robot trunk) or to utilize a joint velocity range close to the hardware limit. In such cases, guiding the search space to the desirable region is required, and a natural way to do this is via constraints.

The constrained Markov decision process (CMDP) framework [36] is a well-known and well-researched model for RL with constraints, where agents must learn to maximize reward while satisfying multiple constraints defined as the expectation of the cost terms. We recommend readers who are not familiar with the CMDP framework or constrained RL to first read Section III, which explains them in detail.

Constrained RL methods can be subdivided into two categories: *Optimization criterion* and *exploration process* [37]. The optimization criterion methods propose policy update rules to satisfy constraints, and the exploration process methods utilize additional safe policies to intervene during the exploration when the agent's state is unsafe. There are several prior works on robotics leveraging exploration process methods for safe exploration when directly trained in real world [38], [39]. In this work, we are focused on optimization criterion methods because safe exploration is not important when the robot is trained in the simulation. The optimization criterion methods can be further divided into two categories based on how the policy is updated: *Lagrangian* and *trust-region* methods.

Lagrangian methods treat the constrained problem as an unconstrained problem by inducing Lagrange multipliers and solving it using primal-dual methods [40], [41]. Although constraints are mostly satisfied when the policy converges, they are sensitive to hyperparameters (e.g., initialization of Lagrange multipliers) and often show unstable training [42]. Trust-region methods update the policy through linear approximation of the objective within the trust region. This subfield contains a variety of algorithms, which can be further subdivided into first-order methods [43], [44], [45] and higher order methods [42], [46]. Only those algorithms that are directly relevant to our work will be covered in greater detail. Achiam et al. [42] proposed constrained policy optimization (CPO) by extending trust region policy optimization (TRPO) to the CMDP framework

with linear approximations of constraints. Although monotonic policy improvements with constraint satisfaction are empirically shown, the requirement of the second-order derivatives imposes implementation difficulty and computation burden. First-order methods alleviate this problem by converting the constrained problem into an unconstrained problem and using first-order gradients to optimize the policy. Different kernel functions are used for stable constraint satisfaction. Liu et al. [43] proposed interior-point policy optimization (IPO), which uses logarithmic barrier functions for the constraint functions to optimize the policy within the constraint-satisfying region. Zhang et al. [44] proposed penalized proximal policy optimization (P3O), which incorporates clamping functions for the constraint functions to impose a linear cost gradient penalty up until the constraint is satisfied and ignore the cost gradient inside the constraint-satisfying area.

Our policy optimization algorithm is based on IPO but several improvements are added to make the algorithm scalable for multiple constraints. First, an adaptive constraint thresholding method is used to appropriately set the constraint limits based on the current policy's performance and guide the policy to the constraint-satisfying region using steep gradients of logarithmic barrier functions. Second, a multihead cost value function is proposed to parallelize cost value function training and cost advantage computation in a memory-efficient manner. With these improvements, we demonstrate that our framework is capable of training controllers for a variety of legged robots, which necessitates the incorporation of more than ten constraints. Furthermore, we showcase the robustness of these controllers in real-world scenarios. Our strong experimental results demonstrate that our method is superior for real-world robotic applications when compared with the prior works [42], [43] that were tested on simulated benchmark tasks [47] with one or two constraints. There is a previous work by Gangapurwala et al. [10] that defined the problem of training locomotion controllers for legged robots in a CMDP formulation. However, their framework required cost coefficient engineering because they handled several constraints as rewards with appropriately set coefficients. Furthermore, their learning framework required considerable reward engineering due to multiple reward terms and a reference trajectory generator, whereas our framework requires only three reward terms and no demonstration data. Our comparison experiment with [10] also demonstrates that the controller obtained with our method is superior in the perspective of safety and task performance.

III. BACKGROUND

A. Constrained Markov Decision Process

CMDP [36] is an augmented MDP framework that is used to define a constrained RL problem. A CMDP is defined as $(S, A, P, R, C_{1,\dots,K}, \rho, \gamma)$, where S denotes the state space, A denotes the action space, $P : S \times A \times S \mapsto \mathbb{R}^+$ is the transition model, $R : S \times A \mapsto \mathbb{R}$ is the reward function, $C_k : S \times A \mapsto \mathbb{R}$ is the cost function for $\forall k \in \{1, \dots, K\}$, ρ is the initial state distribution, and γ is the discount factor. Let $J(\pi)$ and $J_{C_k}(\pi)$ denote the expected discounted return of policy π with respect

to the reward and cost functions as defined as follows:

$$J(\pi) := \mathbb{E}_{\rho, \pi, P} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right]$$

$$J_{C_k}(\pi) := \mathbb{E}_{\rho, \pi, P} \left[\sum_{t=0}^{\infty} \gamma^t C_k(s_t, a_t, s_{t+1}) \right]. \quad (1)$$

The constrained RL problem can be defined as finding a policy that maximizes $J(\pi)$ and satisfies all constraints $J_{C_k}(\pi)$ as follows:

$$\pi^* = \arg \max_{\pi \in \Pi_{\theta}} J(\pi)$$

$$\text{s.t. } J_{C_k}(\pi) \leq d_k \quad \forall k \in \{1, \dots, K\} \quad (2)$$

where d_k is the threshold for the k th constraint, and Π_{θ} is the set of parameterized policies with parameters θ . The set of policies that satisfy all constraints is denoted as the *feasible region*. According to the derivation done by Achiam et al. [42] and Schulman et al. [48], the constrained RL problem (2) can be approximated inside the trust region as below:

$$\pi_{i+1} = \arg \max_{\pi \in \Pi_{\theta}} \mathbb{E}_{s \sim d^{\pi_i}} [A^{\pi_i}(s, a)]$$

$$\text{s.t. } J_{C_k}(\pi_i) + \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi_i}} [A_{C_k}^{\pi_i}(s, a)] \leq d_k \quad \forall k$$

$$\bar{D}_{KL}(\pi || \pi_i) \leq \delta \quad (3)$$

where d^{π} is the discounted state distribution of policy π , defined by $d^{\pi}(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t P(s_t = s | \pi)$, A^{π} is the reward advantage function, $A_{C_k}^{\pi}$ is the k th cost advantage function, $\bar{D}_{KL}(\pi || \pi_i) = \mathbb{E}_{s \sim d^{\pi_i}} [D_{KL}(\pi || \pi_i)[s]]$, and $\delta > 0$ is the maximum step size. The optimal safe policy for the approximated constrained RL problem (3) can be obtained by various algorithms [42], [43], [44], [45], [46].

IV. LEARNING FRAMEWORK

A. Constraint Types

The policy search space is typically too large when training a neural network policy from scratch to control a complex articulated system, making it difficult to design a policy that yields desired motions. Designing advanced reward signals with fine-grained reward engineering can be a solution [1], [6], [9], [21], but the process is highly time-consuming and robot-specific. In this work, we rather leverage constraints to restrict the search space and train the policy within the constraint-satisfying regions.

In RL, stochastic policies are often used [48], [49] for better exploration. Thus, constraints for the policies should be defined in a probabilistic manner, which is the biggest difference from deterministic constraints commonly used in numerical optimization problems. We use two types of constraints: *probabilistic* and *average* constraints. Each constraint can be customized in a form that suits the engineer's intent.

1) *Probabilistic Constraints*: A probabilistic constraint is used to limit the probability of an undesirable event. It can be set

by defining the cost function as an indicator function as below:

$$C_k(s, a, s') = \begin{cases} 0, & \text{if } (s, a, s') \in \mathbf{S} \\ 1, & \text{otherwise} \end{cases} \quad (4)$$

where $\mathbf{S}$ is the robot's desirable event space. Then, the constraint is written as follows:

$$\text{Prob}((s, a, s') \notin \mathbf{S}) = \mathbb{E}_{\rho, \pi, P} [C_k(s, a, s')] \leq D_k \quad (5)$$

where $D_k \in [0, 1]$ is the probability threshold for the k th constraint. If D_k is set to zero, the policy is trained to completely stay away from encountering the undesirable event. Consequently, the probabilistic constraint can be employed to establish constraints for the undesirable region that the robot should avoid at every time step. For example, a robot's center of mass (COM) position should remain within a specific distance threshold from the ground to pass through the overhanging obstacles. We can define the desired area $\mathbf{S}$ for the COM and set the limit of the probability constraint to zero. However, satisfying the constraint of $D_k = 0$ in stochastic MDP settings is challenging. Thus, in our work, D_k is instead set to a fairly small value.

2) *Average Constraints*: An average constraint is used to restrict the average of some physical variables of the robot to be below a desired threshold. It can be set by defining the cost function as below:

$$C_k(s, a, s') = f(s, a, s') \quad (6)$$

where $f(s, a, s')$ is the corresponding physical variable. Then, the constraint is written as

$$\mathbb{E}_{\rho, \pi, P} [f(s, a, s')] = \mathbb{E}_{\rho, \pi, P} [C_k(s, a, s')] \leq D_k \quad (7)$$

where D_k is the value threshold for the k th constraint. This type is appropriate when the goal is to limit the average value rather than to enforce constraints at every step. For instance, it is preferable for legged robots to have low foot velocities when making contact with the ground to avoid slippage. Because the foot velocity may not be zero on slippery surfaces, it is undesirable to use a probabilistic constraint. Instead, we can specify a threshold below which the average foot contact velocity should be.

B. Policy Optimization

The probabilistic and average constraints are handled as discounted cumulative constraints during the policy optimization as follows:

$$J_{C_k}(\pi) := \mathbb{E}_{\rho, \pi, P} \left[\sum_{t=0}^{\infty} \gamma^t C_k(s_t, a_t, s_{t+1}) \right] \leq d_k \quad (8)$$

where $d_k = D_k / (1 - \gamma)$ is the modified constraint limit. This has been done in previous works [42], [43] to efficiently optimize the policy in CMDP formulation (2) with a near-constraint satisfaction guarantee [42].

We use the IPO [43] to solve the constrained RL problem (2). IPO converts the constrained problem into an unconstrained

problem using logarithmic barrier functions as follows:

$$\underset{\pi \in \Pi_\theta}{\text{maximize}} J(\pi) + \sum_{k=1}^K \log(d_k - J_{C_k}(\pi))/t \quad (9)$$

where $t > 0$ is the hyperparameter that determines the steepness of the logarithmic barrier functions. With some approximations (3), the IPO objective (9) is modified as follows:

$$\begin{aligned} & \underset{\pi \in \Pi_\theta}{\text{maximize}} \mathbb{E}_{\substack{s \sim d^{\pi_i} \\ a \sim \pi}} [A^{\pi_i}(s, a)] \\ & + \sum_{k=1}^K \log \left(d_k - \left(J_{C_k}(\pi_i) + \frac{1}{1-\gamma} \mathbb{E}_{\substack{s \sim d^{\pi_i} \\ a \sim \pi}} [A_{C_k}^{\pi_i}(s, a)] \right) \right) / t \\ & \text{s.t. } \bar{D}_{KL}(\pi || \pi_i) \leq \delta. \end{aligned} \quad (10)$$

In the original paper [43], the IPO objective (10) is optimized with proximal policy optimization (PPO) [49]. However, we utilized TRPO [48] because the optimizer's explicit line search allows for more stable policy improvement and constraint feasibility checking.¹ The advantage functions A^π and $A_{C_k}^\pi$ are computed using generalized advantage estimation (GAE) [50] and neural network value functions. Each advantage function then undergoes additional standardization and zero-mean normalization before computing the policy gradients.

Because we are utilizing constrained RL to train controllers for complex robotic systems, the method should be capable of dealing with a large number of constraints. However, earlier publications in the literature, such as CPO and IPO, had limits when it came to explicitly addressing these issues. To that purpose, we incorporate the following two modifications to the IPO to make it scalable for the number of constraints.

1) *Adaptive Constraint Thresholding*: IPO requires the policy to be located in the feasible region during the training because it uses logarithmic barrier functions. However, this setting is not practical in situations where policies are randomly initialized. For these circumstances, a naive approach [45] can be utilized, in which one of the violated constraints is chosen and the policy is adjusted until it enters the feasible region by directly reducing the cost advantage functions. This, however, makes the training inefficient because updating the violated constraints one at a time becomes difficult as the number of constraints increases.

In this work, we widen the feasible region by adaptively adjusting the constraint limit based on the performance of the current policy π_i . Specifically, the k th constraint limit for the i th policy update d_k^i is set as

$$d_k^i = \max(d_k, J_{C_k}(\pi_i) + \alpha \cdot d_k) \quad (11)$$

where $J_{C_k}(\pi_i) = \mathbb{E}_{\rho, \pi_i, P} [C_k(s, a, s')]/(1-\gamma)$, and α is the hyperparameter that determines how much the feasible region will be enlarged.

This makes the problem always feasible due to zero-mean normalization done for the cost advantage function $A_{C_k}^\pi$ and brings a similar effect as scale normalization by parameterizing

¹We also experimented with the PPO [49] optimizer and found several algorithmic advantages, such as ease of implementation and the availability to utilize recurrent neural networks, albeit with the tradeoff of training stability.

Algorithm 1: Modified IPO.

Input: Policy π^{θ_0} , Value function V^{ϕ_0} , and Multi-head Cost Value function $V_{C_{1..K}}^{\psi_0}$. Hyperparameter t for logarithmic barrier functions and α for adaptive constraint thresholding.

Output: Policy π^{θ_N}

- 1: **for** iteration $n=0,1,\dots,N$ **do**
 - 2: Sample a set of trajectories $\{\tau\} \sim \pi^{\theta_n}$ consisting of tuples $(s, a, r, c_1, \dots, c_K)$
 - 3: Compute advantage function A^{π_n} and cost advantage functions $A_{C_{1..K}}^{\pi_n}$ using GAE
 - 4: Update constraint thresholds with (11)
 - 5: Update policy parameters θ with (10)
 - 6: Update value function parameters ϕ and multi-head cost value function parameters ψ
 - 7: **end for**
 - 8: **return** policy parameters $\theta = \theta_N$
-

the enlarging range with d_k . Furthermore, the steep inclination of a barrier function gradually guides the policy toward the desired constraint region defined with threshold d_k .

2) *Multihead Cost Value Function*: For each constraint, parameterized cost value functions (in our case, neural networks) are required to compute the cost advantage function with GAE [50]. When there are multiple constraints, one implementation choice is to use independent neural networks with no shared parameters for each cost value function. However, the number of parameters to train increases linearly with the number of constraints. Instead, we employ a multihead cost value function, in which a single neural network predicts all cost values rather than just one. In this setting, all cost value functions share the same neural network backbone, and only the output layer dimension grows when more constraints are introduced.

The overall policy optimization procedure to solve the RL problem with multiple constraints (2) is summarized in Algorithm 1.

V. APPLICATIONS ON LEGGED ROBOT LOCOMOTION

We applied the proposed learning framework, consisting of both rewards and constraints, to train locomotion controllers for legged robots. Designing robust controllers for legged robots is challenging due to the underactuated nature and the complexity of the hardware. Recently, various works [6], [8], [9], [21] used model-free deep RL to design controllers and demonstrated impressive control performance in perspective of both robot's speed and robustness. However, the reward-only framework, which they utilized, required a lot of effort in reward engineering (more than ten reward terms were designed and the reward coefficients for each of them were tuned laboriously) to create a natural motion. In this work, we train high-performance locomotion controllers with significantly fewer reward terms to engineer by leveraging more intuitive and generalizable constraints.

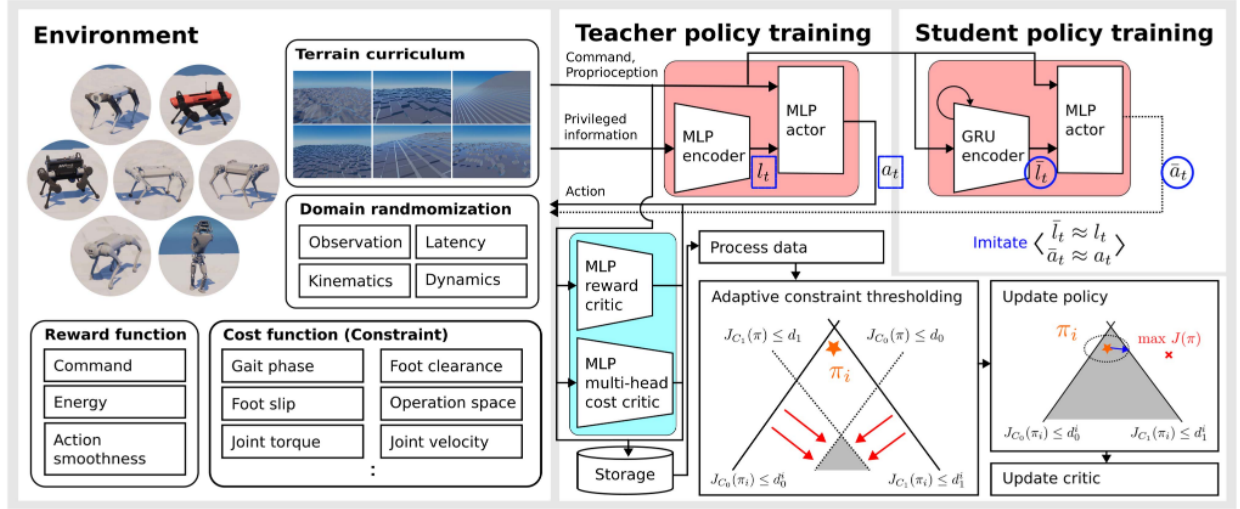


Fig. 2. Overall framework that leverages both rewards and constraints to train locomotion controllers for various legged robots.

A. Overview

Our objective is to devise a blind locomotion controller for legged robots, enabling them to navigate demanding terrains, as exemplified by the work of Lee et al. [6]. Blind locomotion controllers solely depend on proprioceptive sensor data (e.g., joint positions and body orientation) to adjust base motions and foot placements. The controller is given a velocity command, which consists of the desired forward velocity (m/s), lateral velocity (m/s), and turning rate (rad/s), and should track them as closely as possible even under unexpected terrain variations.

Our training pipeline is built based on the current state-of-the-art works [6], [21] that leverage teacher-student learning in the physics simulation and transfer the policy to the real world in zero-shot. In these works, the teacher policy is first trained with model-free RL using proprioceptive sensor information and privileged information only available in the simulation. The student policy is subsequently trained to mimic the behaviors of the teacher policy and forecast compressed privileged information based on the history of proprioceptive sensor data. The student network is trained by supervised learning. We use the proposed learning framework consisting of both rewards and constraints for training the teacher policy. Unlike the previous works that represented hardware constraints and motion styles as rewards, we directly incorporate them as constraints into our problem formulation. We found that only three reward terms are sufficient to obtain a natural gait policy that performs on par with the state-of-the-art policy.

Broad interaction data are crucial to make the controller robust against various rough terrains in the real world. To this end, parameterized terrains with various geometries are procedurally generated during training. The terrains are generated in a way that is not too difficult for the current policy to learn [51]. Domain randomization [23] is done during training for some possible physical properties that can cause a sim-to-real gap. The overall learning framework is summarized in Fig. 2.

B. Teacher Policy

We formulate the control problem as a CMDP. In this subsection, we provide the core elements of the CMDP for teacher training consisting of state space, action space, reward function, and cost functions.

The state space is defined as $s_t := \langle o_t, p_t \rangle$, where o_t is the observation directly accessible in the robot from various sensors and p_t is the privileged information only available in the physics simulation. o_t contains the given velocity command, body orientation, body angular velocity, current joint positions and velocities, joint position errors and velocities measured at $(-0.02, -0.04, \text{ and } -0.06 \text{ s})$, action histories at $(-0.01 \text{ and } -0.02 \text{ s})$, and central phases for each leg. Central phases, defined as cosine and sine pairs $(\cos(\Phi(t)), \sin(\Phi(t)))$ of internal clocks $\Phi(t) = 2\pi ft + \Phi_0$ of the robot, are used to depict desired gait patterns by altering gait frequency f and initial phase shift Φ_0 . p_t consists of information that is not directly available from the robot's measurements but can be useful for control and be approximately predicted from measurement histories. Specifically, it includes terrain friction coefficient, body height with respect to the ground, body linear velocity, body contact state, foot contact impulse, foot airtime, foot clearance with respect to the ground, and terrain profiles. Terrain profiles are given to the robot in the form of circular height scans near each foot similar to the work by Lee et al. [6].

The action space is defined as joint proportional-differential (PD) targets, which are converted to joint torques by a PD controller module at a higher control frequency (4000 Hz). For stable training, we follow the technique used by Ji et al. [8], where the action space is parameterized with nominal joint positions and an action scaling factor that is set constant during training. Specifically, the desired P target is computed as $q_t^{\text{des}} = q^{\text{nominal}} + \sigma_a \cdot a_t$, where q^{nominal} is the nominal joint configuration, σ_a is the action scaling factor, and a_t is the neural network policy output. D target is set to zero. All the robots that we used in the experiment have an action space dimension of 12 and σ_a is set to 0.4.

TABLE I
REWARD FUNCTIONS

Reward	Expression
Command tracking	$r_c = -k_c(\ \text{cmd}_{v_{xy}} - V_{xy}\ ^2 + (\text{cmd}_{w_z} - w_z)^2)$
Joint torque	$r_\tau = -k_\tau\ \tau\ ^2$
Action smoothness	$r_s = -k_s(\ q_t^{\text{des}} - q_{t-1}^{\text{des}}\ ^2 + \ q_t^{\text{des}} - 2q_{t-1}^{\text{des}} + q_{t-2}^{\text{des}}\ ^2)$

Rewards are composed of three terms: command tracking, joint torque, and action smoothness. Command tracking reward and joint torque reward guide the robot to track the given velocity command with minimal torque usage. Action smoothness reward is additionally used to regulate the neural network policy to have a smooth output surface, which is crucial for preventing motor vibration in a real-world deployment. The detailed reward equations are given in Table I. The final reward given to the robot is $r = r_c + r_\tau + r_s$, where k_c , k_τ , and k_s are reward coefficients representing relative weights between different reward terms. To make the training framework more generalizable across different robot platforms, we used the linear relationship between the energy usage (i.e., torque usage $\|\tau\|^2$) and the robot's mass. To this end, we parameterized the torque reward coefficient k_τ of a robot with mass m as $k_\tau = \hat{k}_\tau \cdot \bar{m}/m = (s_\tau \cdot \bar{k}_\tau) \cdot \bar{m}/m$, where $\hat{k}_\tau$ is the torque reward coefficient before mass compensation, s_τ is the scaling factor, $\bar{k}_\tau$ and $\bar{m}$ each are the torque reward coefficient and the mass of the reference robot, respectively. We first find an adequate torque reward coefficient for the reference robot $\bar{k}_\tau$ by trial and error. For the reference robot, $s_\tau = 1$ and $m = \bar{m}$. When training for a different robot, we initialize the torque reward coefficient to $k_\tau = \bar{k}_\tau \cdot \bar{m}/m$, where $s_\tau = 1$, and modify s_τ as we observe to their resulting motions. In our work, a quadruped robot Raibo [9] is the reference robot.

Cost functions are defined based on their constraint type. If the constraint should be satisfied at every time step or is easier to be represented via probability, we formulate it as a probabilistic constraint (5) and use an indicator function for the cost function (4). If not, the constraint is formulated as an average constraint (7) by defining the cost function as the corresponding physical variable (6).

The following terms are defined as probabilistic constraints.

- 1) *Joint position* (c_{jp}): Limit the upper and lower bounds of each joint angle to be in a feasible or desirable operational space. The cost function is an indicator function giving 0 if the current joint angle is in the desirable range, and $1/n_{\text{joints}}$ if not.
- 2) *Joint velocity* (c_{jv}): Limit the upper and lower bounds of joint velocities according to the specifications of joint motors. The cost function is an indicator function giving 0 if the current joint velocity is in the desirable range, and $1/n_{\text{joints}}$ if not.
- 3) *Joint torque* (c_{jt}): Limit the upper and lower bounds of joint torques according to the specifications of joint motors. The cost function is an indicator function giving 0 if the current joint torque is in the desirable range, and $1/n_{\text{joints}}$ if not.

- 4) *Body contact* (c_{bc}): Avoid contact between terrains and the rest of the body except the feet. The cost function is an indicator function giving 0 if the detected contact is between the terrain and the feet, and 1 if not (e.g., self-collision, contact between the terrain and trunk/hip/thigh).
- 5) *COM frame* (c_{com}): Limit the COM height to a reasonable range from the ground and the COM frame orientation to a desirable range with regard to gravity. The cost function is an indicator function giving 0 if the COM frame is in the desirable range, and 1 if not (e.g., COM height too close to the ground, COM orientation too tilted with respect to the gravity vector).
- 6) *Gait pattern* (c_{gp}): Approximately match the predefined foot contact timing designed by the control engineer (e.g., trot, bound). The contact timing is defined based on the central phases for each leg by setting appropriate f and Φ_0 , where the foot is in a swing phase if $\sin(\Phi(t)) < 0$, and a stance phase if not. The cost function is an indicator function giving 0 if the current foot contact state matches the desired state, and $1/n_{\text{legs}}$ if not.

The desirable range $\mathbf{S}$ defining the indicator cost functions of *joint angle*, *joint velocity*, and *joint torque* constraints can be retrieved automatically from the robot description file (e.g., URDF files) if available, or manually determined by the engineer based on the desired motion to obtain or prior knowledge of the system to control. The constraint limits for the above constraints, except *gait pattern* constraint c_{gp} , are set to a fairly small value to restrict the policy from entering the undesirable region.

Below terms are defined as average constraints.

- 1) *Orthogonal velocity* (c_{ov}): Limit the body velocity in a direction not given in the velocity command (i.e., $v_{\text{ortho}} = (v_z, w_x, w_y)$) to avoid noisy body motions. The cost function is defined as $c_{ov} = \|v_{\text{ortho}}\|_1/3$.
- 2) *Contact velocity* (c_{cv}): Limit the velocity at contact points to prevent slippage. The cost function is the average of all contact point velocities $c_{cv} = \sum_{\forall \text{contact}} \|(v_x, v_y)\|_2 / N_{\text{contact}}$.
- 3) *Foot clearance* (c_{fc}): Limit the foot clearance to be above a desired value. Foot clearance is defined as the foot height from the ground at the peak of a swing phase (i.e., $\sin(\Phi_t) \approx -1$). Foot clearance for each foot is updated at every leg phase period. The cost function is defined as $c_{fc} = -\sum_{\forall i \in \{1, \dots, n_{\text{leg}}\}} D_{\text{leg}}^i / n_{\text{leg}}$, where D_{leg}^i is the foot clearance of leg i . Minus is multiplied because the constraints are defined with upper bounds as in (2).
- 4) *Foot height limit* (c_{fh}): Limit foot height with respect to the ground to prevent raising them too high. The cost function is defined as $c_{fh} = \max(d_{\text{leg}}^1, d_{\text{leg}}^2, \dots, d_{\text{leg}}^{n_{\text{leg}}})$, where d_{leg}^i is the current foot height of leg i with respect to the terrain profile near each foot.

The average constraints have the advantage of allowing the constraint thresholds to be defined intuitively because they directly correspond to physical quantities with known units [e.g., velocity (m/s) and height (m)]. For example, if the control engineer wants the robot to raise the foot higher than 0.1 m, then the threshold of *foot clearance* constraint just needs to be set to -0.1 . Thresholds for the *orthogonal velocity* and *contact*

velocity constraints are set to small values based on the logs obtained during the training.

In addition, we constrain the body motion to be generated symmetrically by utilizing the symmetric loss proposed by Yu et al. [52]. The symmetric constraint c_{sym} is defined as follows:

$$L_{\text{sym}} := \mathbb{E}_{s \sim d^\pi} [\|\mu_\theta(s) - \Psi_a(\mu_\theta(\Psi_s(s)))\|_1] \leq d_{\text{sym}} \quad (12)$$

where Ψ_a and Ψ_s are functions that mirror the action and state with respect to the XZ plane of the base frame, respectively, and μ_θ is the mean of the Gaussian policy π parameterized with neural network parameters θ . The symmetric constraint is handled in the same manner as the average constraint by using the reparameterization trick [53].

To summarize, a total of 11 constraints (six probabilistic constraints and five average constraints) are defined and used to train a control policy for legged robots. To the best of the authors' knowledge, this is the greatest number of constraints ever used in the constrained RL literature. Some of the constraints may not be critical during the optimization depending on the robot and the environment. For example, for some robots, the *joint velocity* constraint may have less of an impact because the desirable range was chosen with enough margin, and so the initial policy already satisfied the desired constraint threshold. However, constraints do not need to be switched ON and OFF according to their usage because of the property of logarithmic barrier functions. Concretely, already satisfied constraints have no effect on policy optimization due to a near-zero gradient of the barrier function. However, the barrier function applies a penalty when the policy tries to exit the desired region during exploration via a steep gradient.

The teacher policy is treated as a Gaussian policy, with the neural network outputs and state-independent trainable parameters corresponding to the distribution's mean and standard deviation. The teacher policy network is constructed with two multilayer perceptron (MLP) blocks: The MLP encoder and the MLP actor. The MLP encoder takes the privileged information p_t as input and encodes it to a latent representation l_t . l_t is then concatenated with the observation o_t and passed to the MLP actor, which outputs the action a_t .

C. Student Policy

The student policy is trained to imitate the teacher policy's behavior. As the student policy is to be deployed in the real world in a zero-shot manner, it should only leverage information directly available from the robot's sensor and cannot utilize the privileged information p_t used for the teacher policy. Thus, the student policy is trained to imitate the teacher policy's action a_t while predicting the encoded privileged representation l_t from observation histories $\{o_0, o_1, \dots, o_t\}$.

For this purpose, the student policy network is constructed with a gated recurrent units (GRU) [54] encoder and an MLP actor. The GRU encoder takes in the observation o_t and the internal hidden state of the unit and predicts the encoded privileged information $\bar{l}_t$. $\bar{l}_t$ is then concatenated with o_t and passed to the MLP actor to output the action $\bar{a}_t$. The network is trained

end-to-end with the loss function $L_{\text{student}}(\theta) = \|\bar{a}_t(\theta) - a_t\|_2^2 + \|\bar{l}_t(\theta) - l_t\|_2^2$, where θ is the neural network parameter of the student policy, to imitate both the teacher's action and the latent privileged representation.

D. Terrain Curriculum

Similar to the previous works [6], [8], [9], [21], broad parameterized terrains are generated in the simulation by sampling the corresponding terrain parameters in a desired range. Five different types of terrain (i.e., hills, discrete hills, steps, inclining steps, and stairs) are sampled in an equal proportion and each type is modeled with two different terrain parameters similar to previous works [6], [21].

During teacher policy training, an adaptive terrain curriculum is held to procedurally generate terrains that can give rich training signals to the network. The method we use is similar to the fixed-order curriculum proposed by Xie et al. [51]. The total ranges for each terrain parameter are divided into N_{stage} and at each iteration, terrains are sampled uniformly between 0 and N_{max}^i stage, where N_{max}^i ($\forall i, N_{\text{max}}^i \in \{0, \dots, N_{\text{stage}}\}$) is the maximum available sampling stage for i th terrain type. N_{max}^i is set to 0 at the beginning of the training and procedurally grows whenever the policy's traversability score at the N_{max}^i stage exceeds the defined threshold. The traversability score is defined by Lee et al. [6], where the value is between 0 and 1. There are more sophisticated terrain curriculum methods available, such as the particle-filter approach [6]. In our experience, however, the strategy was still able to prevent catastrophic forgetting with essentially less related hyperparameter engineering, while still enabling strong policy training with effective terrain exploration.

The terrain curriculum is not employed during student policy training, but rather uniformly sampled in the terrain range for which the teacher policy is trained.

E. Domain Randomization

For the zero-shot sim-to-real transfer of the trained control policy, critical parameters of the robot are randomized during the training. Specifically, observation noise, motor frictions, PD controller gains, foot positions and collision geometry, ground friction, and control latency are randomized in the predefined distribution at the start of the episode or at every time step.

VI. EXPERIMENTAL RESULTS

We conducted several experiments with the task of legged robot locomotion to answer the questions below regarding a learning framework that incorporates both rewards and constraints.

- 1) Can a controller trained with the proposed framework attain a similar robust control performance as prior works developed with the reward-only approach?
- 2) Can the proposed framework be more generalizable across different robot platforms?
- 3) Can the proposed framework make the engineering process of generating desired motions more straightforward and intuitive?

TABLE II
CONSTRAINT SATISFACTION RESULTS

	Limit	c_{jp}	c_{jv}	c_{jt}	c_{bc}	c_{com}	c_{gp}	c_{ov} (m/s)	c_{cv} (m/s)	c_{fc} (m)	c_{fh} (m)	c_{sym}
		0.025	0.025	0.025	0.025	0.025	0.25	0.35	-	-	-	0.1
Sec. VI-B	Raibo	0.015	0.004	0.001	0.002	0.009	0.16	0.32	0.15 [0.2]	-0.11 [-0.07]	0.09 [0.11]	0.07
	HOUND	0.014	0.020	0.000	0.003	0.002	0.17	0.30	0.13 [0.2]	-0.09 [-0.07]	0.07 [0.11]	0.07
	GoI	0.012	0.007	0.000	0.008	0.001	0.16	0.33	0.10 [0.2]	-0.08 [-0.05]	0.06 [0.09]	0.07
	Mini-cheetah	0.015	0.000	0.001	0.002	0.001	0.13	0.32	0.10 [0.2]	-0.08 [-0.05]	0.06 [0.09]	0.06
	ANYmal B	0.016	0.019	0.000	0.002	0.005	0.16	0.31	0.13 [0.2]	-0.09 [-0.07]	0.07 [0.11]	0.07
	ANYmal C	0.019	0.021	0.002	0.004	0.005	0.16	0.30	0.11 [0.2]	-0.08 [-0.07]	0.06 [0.11]	0.07
	ATLAS	0.011	0.019	0.004	0.008	0.008	0.12	0.29	0.35 [0.5]	-0.17 [-0.15]	0.12 [0.2]	0.08
Sec. VI-C	Torque (x2)	0.017	0.005	0.001	0.001	0.008	0.15	0.33	0.16 [0.2]	-0.13 [-0.07]	0.10 [0.11]	0.08
	Smoothness (x2)	0.013	0.005	0.001	0.001	0.009	0.14	0.32	0.15 [0.2]	-0.13 [-0.07]	0.10 [0.11]	0.07
Sec. VI-E	P3O [10], [44]	0.025	0.005	0.000	0.002	0.016	0.24	0.35	0.2 [0.2]	-0.08 [-0.07]	0.09 [0.11]	0.1
	$t = 100, \alpha = 0.002$	0.015	0.004	0.002	0.003	0.009	0.17	0.32	0.15 [0.2]	-0.11 [-0.07]	0.09 [0.11]	0.07
	$t = 100, \alpha = 0.2$	0.015	0.004	0.002	0.002	0.011	0.18	0.44	0.17 [0.2]	-0.11 [-0.07]	0.09 [0.11]	0.07
	$t = 10, \alpha = 0.02$	0.004	0.001	0.000	0.000	0.002	0.11	0.24	0.07 [0.2]	-0.14 [-0.07]	0.07 [0.11]	0.05
	$t = 1000, \alpha = 0.02$	0.023	0.010	0.002	0.002	0.015	0.22	0.45	0.2 [0.2]	-0.09 [-0.07]	0.09 [0.11]	0.09

*Results are after training with 1.6 billion time steps. Constraint thresholds are reported in the top row or inside the bracket. Bolded texts indicate violated or barely satisfied constraints. Bold values indicate the best values for each evaluation metric.

The rest of this section is organized as follows. After providing some implementation details in Section VI-A, the results of training locomotion controllers for various legged robots are shown in Section VI-B. Some crucial characteristics required for the usage of the learning framework are analyzed in Section VI-C. In Section VI-D, the training performance is compared with the reward-only framework. Finally, ablation studies are conducted in Section VI-E for an elementwise in-depth study of the proposed framework.

A. Implementation Detail

The proposed learning framework was implemented with PyTorch [55]. To train locomotion controllers for legged robots in the physics simulator, the simulator should be capable of producing a large volume of realistic interaction data in a highly parallel and rapid manner. For this purpose, RaiSim [56] physics simulator was utilized. For training, we used AMD Ryzen9 5950X and a single NVIDIA GeForce RTX 3070. It took about 24 h to train the teacher policy and 8 h to train the student policy.

The locomotion control policy runs at 100 Hz. We deployed the trained controller on two real-world quadruped robots: Raibo [9] and Mini-Cheetah [57]. For the deployment, the student policy network was reimplemented with the Eigen library in C++ and forward-passed in the central processing unit. We recommend readers to check the original paper [9], [57] for more details about the robot hardware.

We analyzed the performance of the proposed learning framework in various aspects, such as algorithm computation time, and obtained rewards. The Raibo robot is used to produce the majority of simulation results; if another kind is used, it is noted.

B. Framework Evaluation

The proposed learning framework was used to train locomotion controllers for diverse legged robots in the physics simulator: Raibo [9], Mini-Cheetah [57], HOUND [58], ANYmal B, ANYmal C [59], Unitree Go1, and ATLAS (see Fig. 1). Robots used for training vary in both morphology and physical properties. In particular, two robot types (quadruped and bipedal) are studied, with mass ranging from 11 to 135 kg, and robot

TABLE III
REWARD COEFFICIENTS FOR SEVERAL LEGGED ROBOTS

	k_c	$\hat{k}_\tau$	k_s
Raibo	10.	0.005	0.06
Hound	10.	0.0025	0.06
GoI	10.	0.025	0.06
Mini-cheetah	10.	0.03	0.06
ANYmal B	10.	0.005	0.06
ANYmal C	10.	0.003	0.06
ATLAS	10.	0.001	0.06

size and leg length ranging from a 0.3 m tall small-scale dog with a 0.4 m leg length to a 1.9 m tall person with a 0.9 m leg length. Furthermore, suitable PD gains for each robot varied substantially depending on the robot's motor specification and inertia, and the hind legs design of quadruped robots (ANYmal B and ANYmal C) differed depending on whether they are bent forward or backward.

Regardless of these variations, robust locomotion controllers for individual robots could be trained successfully using our framework with minimal parameter modifications. As shown in Fig. 3(a) and Table II, policies gradually learned to maximize the designed rewards while satisfying all of the constraints. Scaling factor s_τ , which defines the torque reward coefficient k_τ , was only adjusted among the reward coefficients for different robots (see Table III). This is reasonable because energy usage is a complex function that cannot be solely modeled by mass; it is also influenced by factors, such as link inertia and actuator characteristics.

Constraint limits and desirable region $\mathbf{S}$ for some probabilistic constraints were adjusted based on the robot's morphology and size. $\mathbf{S}$ for the *joint angle* constraints were set semantically the same for all quadruped robots. In the case of ATLAS, a bipedal robot, the desirable joint angle ranges $\mathbf{S}$ for the hip joints were manually set in the *joint angle* constraint based on the related online video footage (e.g., the robot's locomotion videos) to prevent the robot from spreading its legs too far to the sides while walking. In the *gait pattern* constraint, the size of the initial phase offset parameter Φ_0 for quadrupedal robots was four (e.g., $[0, \pi, \pi, 0]$), while it was two (e.g., $[0, \pi]$) for bipedal

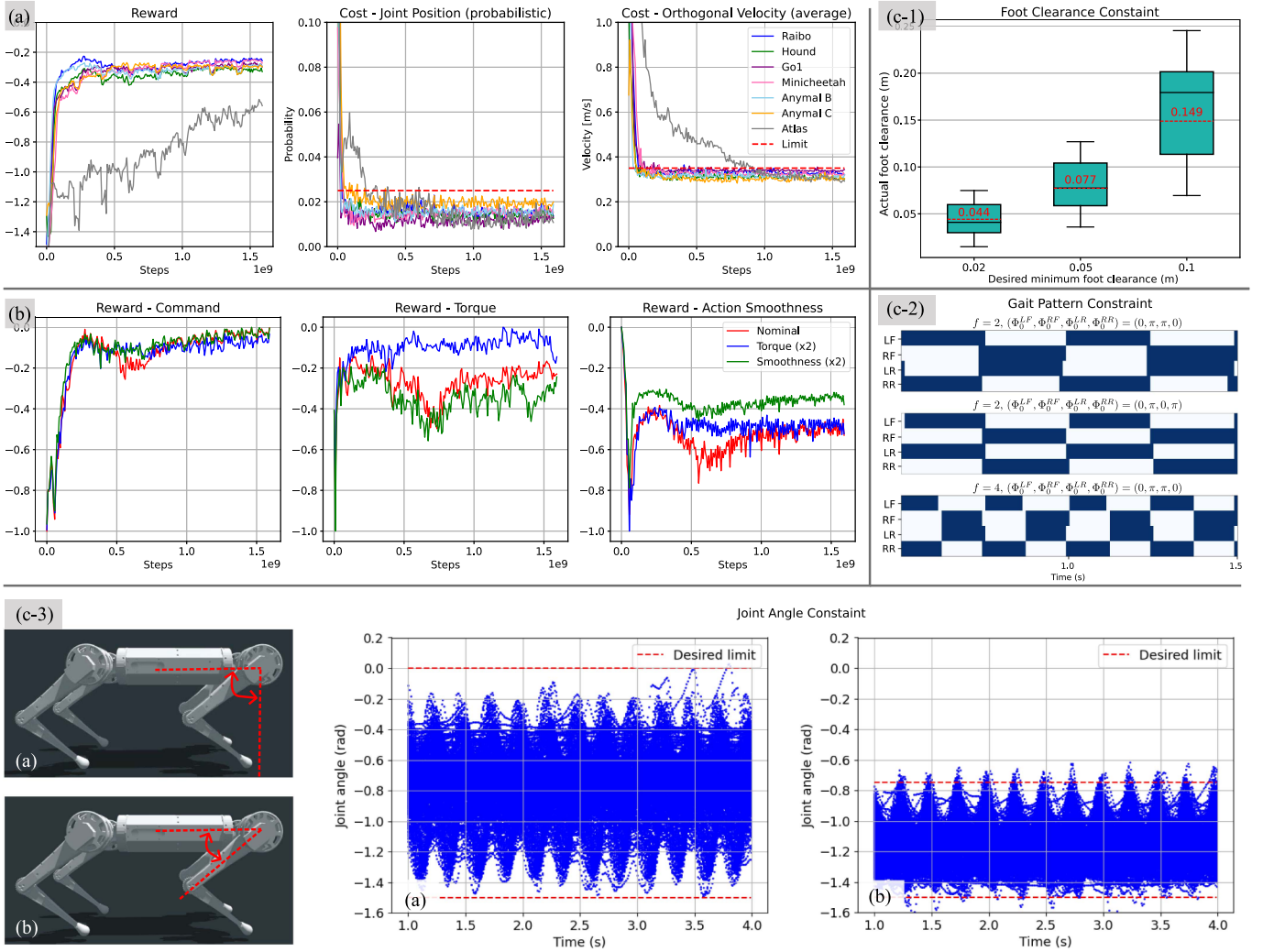


Fig. 3. Evaluation and analysis results of the proposed framework. (a) Total reward (left-hand side) and two costs (middle and right-hand side) of multiple robots. (b) Each reward component's value to check the *objective sensitivity* property. Every reward has been standardized so that it ranges from -1 to 0 . (c) Results to verify the *constraint satisfaction* property. Specifically, foot clearance (c-1), gait pattern (i.e., trot, pace, fast trot) (c-2), and joint angle (c-3) constraints were modified and tested in the simulation. The resulting foot clearances, foot contact patterns, and joint angle trajectories are plotted.

robots. This is necessary because each robot has a different number of legs. Thresholds for the *COM*, *foot clearance*, and *foot height limit* constraints were adjusted based on the size of the robot. For instance, the 7 cm foot clearance limit used for ANYmal C is too large for a small-scale robot, such as Mini-Cheetah. The *contact velocity* constraint threshold was set the same for all quadruped robots but was slightly increased for ATLAS. This was necessary because the foot's collision body of the quadruped robots was a sphere, whereas that of ATLAS was a box. Desirable regions for *joint velocity* and *joint torque* constraints were set automatically from the robot description file (e.g., URDF files). Other constraint limits, desirable regions S , and reward functions were set exactly the same for all robots. Specific limits for each constraint are provided in Table II.

The trained controllers were deployed on two real-world quadruped robots available in the lab: Raibo and Mini-Cheetah. The controllers were extensively tested in various harsh terrains

both indoor and outdoor to verify the robust control performance as shown in the previous work [6]. The testing environments included slippery hills, steep slopes, deformable terrains, stairs, a pile of leaves with hidden impediments, a moving cart, unstably piled planks, and even more. No matter how difficult the terrains were, Raibo and Mini-Cheetah were able to walk steadily by modifying base motions and foot placements based solely on proprioceptive sensor data, as illustrated in Fig. 4. Furthermore, Raibo could sprint over grassy terrain at 2 m/s and climb slippery and steep slopes with an angle of around 32° at 1 m/s, which is roughly double the speed reported in the prior work [6].

Real-world constraint satisfaction was also verified for those that can be directly observed by sensors (i.e., joint angle, joint velocity, and joint torque). For verification, we collected approximately 25 min of real-world robot sensor data with the Raibo robot in the outdoor experiment routes, as shown in Fig. 4. *Joint angle*, *joint velocity*, and *joint torque* constraints are all probabilistic constraints, and their cost values were 0.007, 0.001,

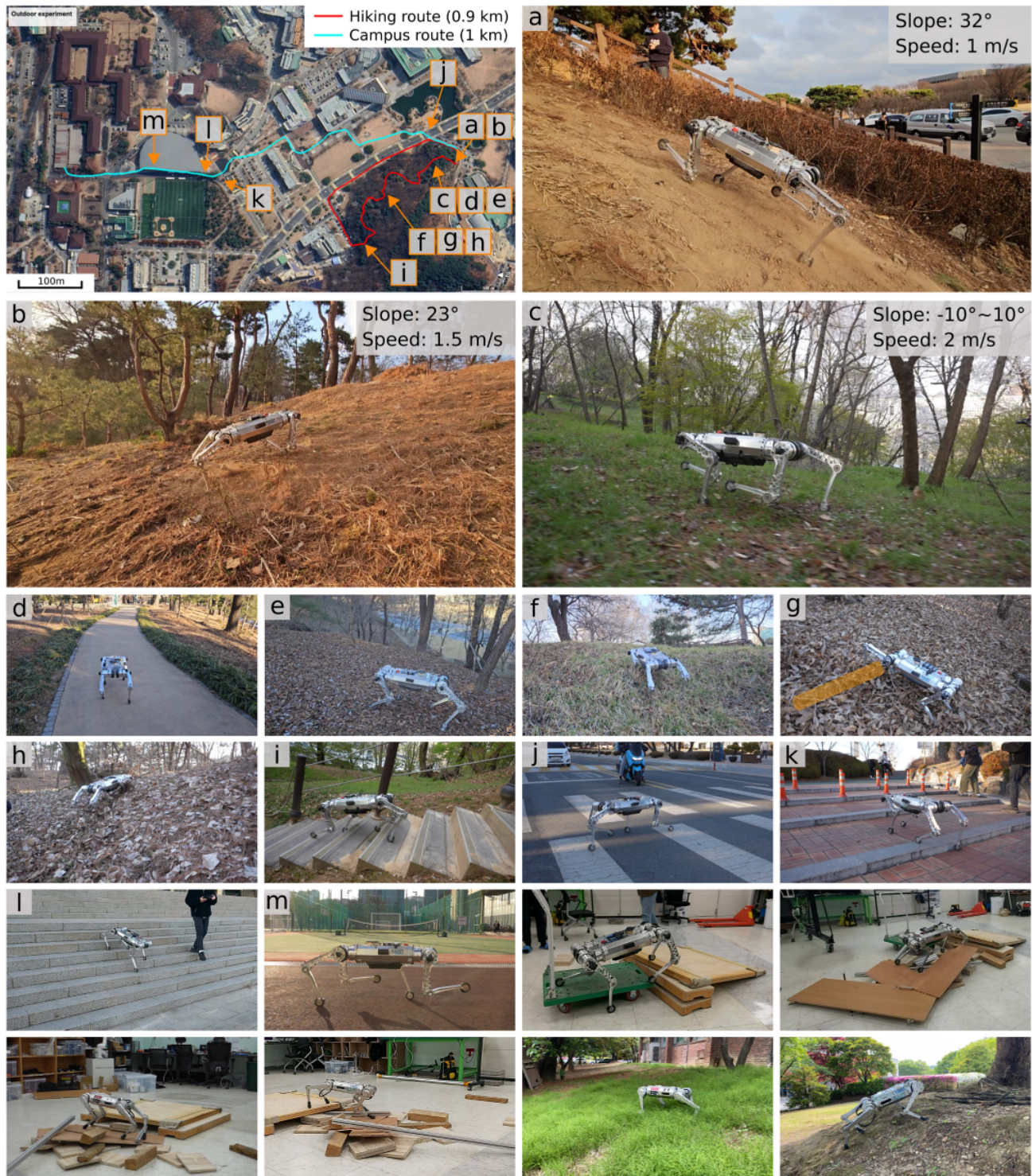


Fig. 4. Real-world experiments with two quadruped robots: Raibo and Mini-Cheetah. The robustness of the controllers was extensively tested in various harsh terrains both indoor and outdoor. Outdoor experiments were conducted on two long paths: a hiking route and a campus route. The robot's speed is calculated using the video's recorded time stamp and the measured length of the traversal path. The slope of the environment is measured using a digital angle gauge.

and 0.002, respectively, indicating constraint satisfaction given that the thresholds were all 0.025.

In summary, locomotion controllers for various legged robots were trained with our framework that utilized only three rewards and generalizable constraints. A single locomotion learning framework could train control policies for N robots with a

minimal amount of engineering. Specifically, only a single reward coefficient was adjusted among the three designed reward terms for robot transfer. If the robot's training is solely guided by a composition of abundant reward terms, the interdependency between them makes the additional engineering process difficult and cumbersome. More analysis about the reward-only



Fig. 5. Real-world experiments of generating multiple motions by varying constraints. A robot walking with a trotting gait (top) was changed to locomote with a bounding gait (middle) or crawl beneath an overhanging obstacle (bottom) by modifying the gait pattern constraint or COM height with joint angle constraint.

framework is provided in Section VI-D. There were some modifications for the constraints, but this can be considered a much more intuitive process for the control engineer compared with the reward coefficient tuning because the constraint bounds all have exact physical meanings. Our extensive real-world experiments show that dynamic and agile controllers trained in simulation can be successfully transferred to the real world in zero-shot and show highly robust performance in various harsh terrains.

C. Framework Analyzation

The proposed learning framework leverages both rewards and constraints, where these two representations are used for different purposes. Rewards are representations of a value to be maximized and can be thought of as an objective function in numerical optimization problems. Constraints, on the other hand, are representations in which engineers express the desirable area that the converged policy should satisfy. To leverage both rewards and constraints for designing neural network controllers, the proposed framework should satisfy below two properties.

- 1) *Objective sensitivity*: By varying the reward coefficients, which are comparable to altering the relative weights of various objective functions, the policy training should show responsive results.
- 2) *Constraint satisfaction*: Regardless of the variations of objectives and constraints, the converged policy should satisfy defined constraints if there exists a policy that satisfies all constraints.

We conducted several experiments to verify these properties. First, we modified objectives by applying a constant scaling factor of two to coefficients of torque reward and smoothness reward, wherein the values are not bounded, and analyzed the policy training results. This is frequently done when engineers

want to emphasize a specific term more when constructing the objective function, such as giving higher weights to the torque usage term to minimize them more. As shown in Fig. 3(b), the policy's specific reward term grew as the corresponding coefficient increased. In addition, the constraints were all satisfied regardless of the modifications of the reward signals (see Table II). This is a promising result for the learning framework because the policy is trained to be responsive to the variation of the objective while satisfying the already existing constraints, which are both defined by the engineer's intent.

Second, we varied the constraints and saw aspects of the converged policies. Desirable regions $\mathbf{S}$ and constraint limits were each modified for the probabilistic constraint and the average constraint. Mini-CheetaH robot was used for this experiment. As shown in Fig. 3(c), varying the constraints guided the policy to different behaviors that belong in the defined region. Specifically, the robot was trained in the simulation to have various foot clearance, walk in different gait patterns, and operate in a different joint angle range by changing the constraints. The effectiveness of using constraints was further analyzed by employing the controllers with several motions in real-world scenarios (see Fig. 5). The robot was trained to crawl below an overhanging obstacle or utilize a bounding gait rather than a trotting gait. For the robot to crawl below an overhanging obstacle, the COM height range for the *COM* constraint and the hip joint range for the *joint angle* constraint were modified. For the robot to locomote with a bounding gait, the initial phase offset parameter Φ_0 for the *gait pattern* constraint was changed to represent them. The above two motions were generated from the exactly same parameter set that was used to train a walking controller with a trotting gait.

As a result, our framework satisfies two important properties, *objective sensitivity* and *constraint satisfaction*, required for training neural network controllers. Based on the *constraint*

TABLE IV
NUMBER OF REWARD TERMS

Lee et al. [6]	6
Miki et al. [21]	9
Kumar et al. [24]	10
Ji et al. [8]	11
Choi et al. [9]	14
Ours	3

Bold values indicate the best values for each evaluation metric.

satisfaction property, engineers can use constraints to restrict the area where the policy converges. Thus, they can generate desired motions efficiently by setting appropriate constraint parameters at their intent. Setting constraints is more straightforward than modifying reward coefficients since it directly restricts the search region with hyperparameters that correspond to physical variables with known units. A minimal number of reward terms, as demonstrated in Table IV, can then be designed and fine-tuned based on the *objective sensitivity* property. Check out the Appendix for a more detailed demonstration of our framework's engineering benefits based on the two properties mentioned above. It is illustrated using a simple toy example and a comprehensive comparison of the entire engineering scenario with the reward-only framework (see Table VIII).

D. Comparison With the Reward-Only Framework

We compare the generalizability of our framework and the reward-only framework, which is dominantly used in previous works on legged robot locomotion [6], [8], [9], [21]. The reward terms and hyperparameters used for the reward-only framework are based on the previous works by Ji et al. [8] and Choi et al. [9]. There were a total of twelve reward terms and they were designed for task completion (i.e., command tracking), physical constraints' satisfaction (i.e., undesirable body contact, joint velocity, joint acceleration, joint torque, and action smoothness), and style constraints' satisfaction (i.e., air time, foot slippage, foot clearance, nominal configuration, hip joint usage, and orthogonal velocity). Adequate reward coefficients for each term were laboriously engineered for robust control performance on the Raibo robot while satisfying all physical constraints. TRPO was used for policy optimization.

The learning framework engineered for the Raibo robot was directly applied to the ANYmal C robot, which is twice heavier and has a different leg morphology. None of the reward coefficients were modified for both methods, except the auto-adaptation of the torque reward coefficient k_τ based on the robot's mass. The scaling factor s_τ is set to one for both methods. As illustrated in Fig. 6(a), ANYmal C robot, trained with our framework, exhibited remarkable locomotion capabilities with comparable command tracking performance.² On the other hand, the controller trained with the reward-only framework exhibited poor locomotion performance and significant degradation in its overall command tracking capability. Furthermore,

²Check out the Appendix for a more in-depth demonstration of the Raibo robot's velocity command tracking performance.

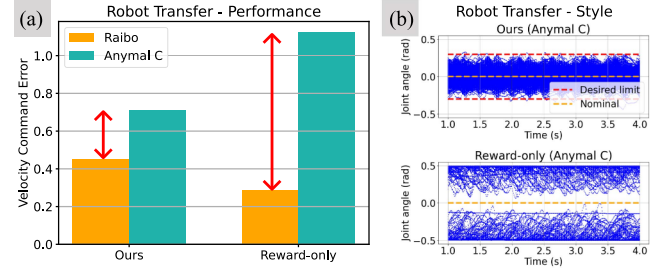


Fig. 6. Comparison results with the reward-only framework in terms of the robot generalizability. Each framework was engineered for robust control performance on the Raibo robot and was directly applied to the ANYmal C robot. (a) Locomotion performance according to the velocity command tracking error $\|\text{cmd}_{v_{xy}} - V_{xy}\|^2 + (\text{cmd}_{w_z} - w_z)^2$. A smaller difference between Raibo and ANYmal C indicates better robot generalizability. (b) Locomotion style of the ANYmal C robot in terms of the hip joint angle trajectories. For the Raibo robot, both frameworks were engineered to utilize joint angle ranges near the nominal joint configuration. A smaller deviation from the nominal joint configuration indicates better robot generalizability.

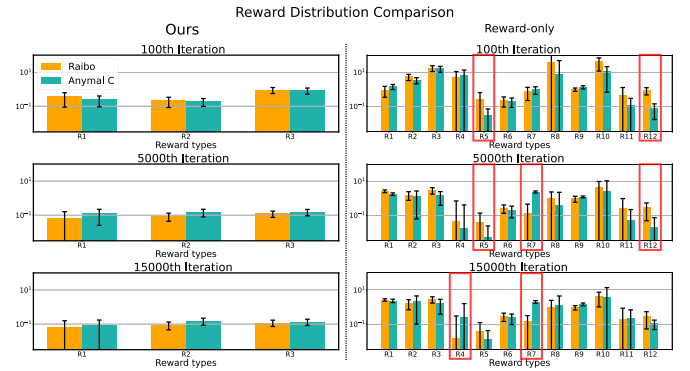


Fig. 7. Comparison of each reward component's distribution with the reward-only framework. Red blocks show reward types with highly different reward distributions.

the converged control policy for the ANYmal C robot utilized joint angle ranges quite apart from the nominal configuration, resulting in poor locomotion style transfer, as shown in Fig. 6(b). These circumstances were prevented for the Raibo robot using an extra reward term (i.e., the nominal configuration reward), which penalizes deviations from the nominal configuration. However, the corresponding reward signal was too small for the ANYmal C robot and the reward coefficients should be adjusted accordingly. In our framework, the locomotion style was more transferable across robots because they were defined as constraints rather than soft penalty terms as used in the reward-only framework.

The poor generalizability of the reward-only framework is because the magnitudes of reward terms change depending on the robot. We plotted the mean and the standard deviation of each reward component for the two robots in Fig. 7. In the reward-only framework, the two robots showed very different reward distributions, thus resulting in different control behaviors. The changes in the reward distributions are due to the differences in the robot's physical properties and morphologies. Consequently, the reward coefficients should be adjusted, but it is not trivial due

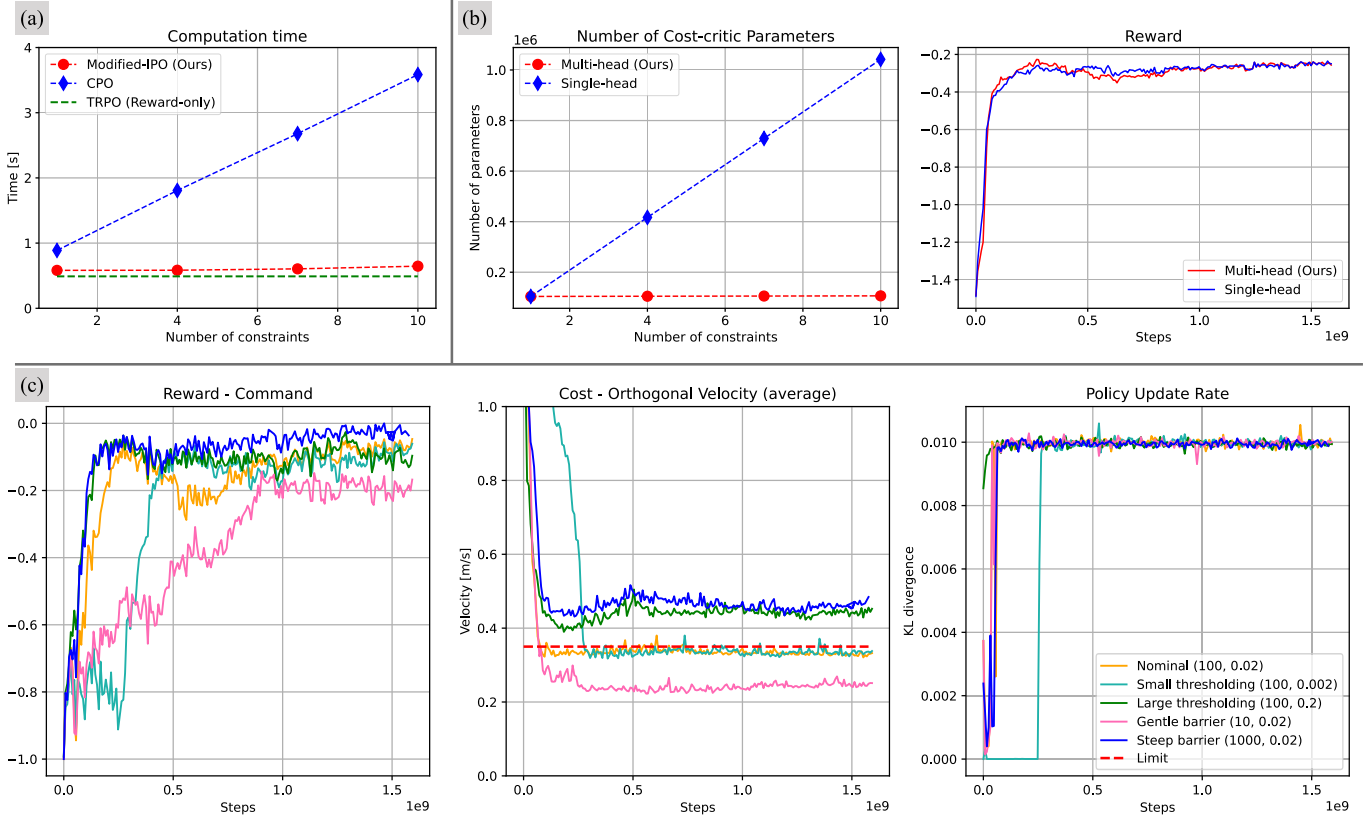


Fig. 8. Ablation study results of the proposed framework. (a) Computation time for different policy optimization algorithms. (b) Number of trainable parameters (left-hand side) and total rewards (right-hand side) for different cost-critic network designs. (c) Hyperparameter t and α sensitivity in terms of the reward (left-hand side), cost (middle), and policy update rate (right-hand side).

to the interdependency among each of the reward components and the reward signal distribution shift over the iteration number. This results in an additional cumbersome reward engineering process that involves a significant amount of trial and error to find the most suitable parameter set.

E. Ablation Study

We conducted several ablation studies to verify some core components of the proposed learning framework. Furthermore, the performance sensitivity to algorithm hyperparameters was analyzed.

1) *Algorithm*: Our policy optimization is based on the IPO [43], which changes the constrained problem to an unconstrained problem using logarithmic barrier functions. We compared the optimization performance with two well-known policy-search algorithms for solving constrained RL problems: CPO [42] and P3O [44]. CPO is an extension of TRPO [48] to the CMDP framework with linear approximations of the cost functions. They convert the primal problem to the dual problem to efficiently find the policy update direction. P3O, similar to IPO, is a first-order method that converts a constrained problem into an unconstrained problem for policy optimization. P3O applies a linear cost gradient penalty until the constraint is satisfied and then ignores the cost gradient within the constraint-satisfying area via the clamping function. Previously, a variant of P3O

was used for training locomotion controllers for legged robots with successful real-world deployment [10]. Check the original papers [42], [44] for more detail.

As illustrated in Fig. 8(a), the computation time for the optimization step grows linearly with the number of constraints when employing CPO. This is because, the number of optimization variables in the dual problem, as well as the number of conjugate gradient methods to be applied, grows in proportion to the number of constraints. If there are one or two constraints, analytic solutions derived from the original paper can be utilized to reduce the computation time. However, we want our learning framework to be scalable enough to handle multiple constraints because it is intended to be used for training neural network controllers for complex robotic systems. IPO is a suitable optimization algorithm to utilize since it exhibits nearly no change in computation time with increasing the number of constraints, and it even has a similar computation time to the optimization step for the reward-only framework (TRPO). This is clear because our framework and the reward-only framework are completely identical when no constraints are given. The major extra computation that occurs when constraints are defined is the computation of the cost advantage functions and training of the cost value functions, which are parallelized using a multihead version.

Because P3O is also a first-order method, it showed a similar computation time as our method and demonstrated that it can be scalable in terms of the number of constraints. However, the

TABLE V
ALGORITHM PERFORMANCE COMPARISON

	Reward (r)	Torque	Smoothness	Traversability
Ours	-0.242 (± 0.007)	24.769 (± 5.811)	0.093 (± 0.002)	0.811 (± 0.055)
P3O [10], [44]	-0.271 (± 0.012)	26.186 (± 6.116)	0.101 (± 0.002)	0.781 (± 0.068)

Bold values indicate the best values for each evaluation metric.

TABLE VI
MULTIHEAD COST-CRITIC EVALUATION (# CONSTRAINTS = 10)

	# parameters	Computation time (s)	Prediction loss
Multihead (Ours)	0.1 M	0.23 (10.22 %)	0.468 (± 0.120)
Single head	1 M	0.56 (24.89 %)	0.435 (± 0.139)
Single head (Reduced)	0.1 M	0.33 (14.67 %)	0.534 (± 0.205)

* Percentage in the “Computation time” indicates the ratio of time spent on cost critic versus others (e.g., simulation), where others are identical for all three methods. Bold values indicate the best values for each evaluation metric.

policy trained with IPO performed better than those trained with P3O (see Table V). The policy trained with IPO showed higher reward, smaller torque usage (i.e., smaller energy usage), better action smoothness (i.e., smaller motor vibration for real-world deployment), and a higher traversability score [6] (i.e., better ability to traverse diverse terrains at high speed). Furthermore, unlike the policy trained with IPO, which showed clear constraint satisfaction, the policy trained with P3O converged near the constraint threshold and exhibited constraint violation in some cases (see Table II). This is because IPO takes into account a smooth cost gradient within the constraint-satisfying region in the form of a logarithmic barrier function, whereas P3O completely disregards it. The smooth cost gradient within the constraint-satisfying region enables better policy guidance for high performance (e.g., the smooth gradient derived from the contact velocity constraint penalizes foot slippage even after satisfying the contact velocity constraint) and safety (e.g., the smooth gradient derived from the joint position constraint penalizes joint angle usage near the undesirable range).

2) *Multihead Cost Value Function*: We utilized the multihead cost value function when computing the cost advantage functions using GAE [50]. We compared the number of trainable parameters, computation time, and value prediction loss during training to the two approaches listed below.

- 1) *Single head*: Use independent neural networks with no shared parameters for each cost value function. The same neural network size with the multihead method is applied for each cost value function.
- 2) *Single head (Reduced)*: Use independent neural networks with no shared parameters for each cost value function. The reduced neural network size is applied for each cost value function to match the total number of trainable parameters with the multihead method.

The results are shown in Fig. 8(b) and Table VI. The multihead method requires fewer trainable parameters than the single-head method. The multihead cost value function requires 0.1 million trainable parameters, whereas single-head cost value functions

TABLE VII
VELOCITY COMMAND TRACKING PERFORMANCE COMPARISON

	Velocity command error	Torque
Ours + L2 (Raibo)	0.498 (± 0.308)	24.769 (± 5.811)
Ours + Exp (Raibo)	0.367 (± 0.290)	29.489 (± 8.311)
Ours + Exp (ANYmal C)	0.547 (± 0.380)	60.310 (± 14.978)
Reward only (Raibo)	0.359 (± 0.354)	30.773 (± 10.181)

Bold values indicate the best values for each evaluation metric.

require 1 million. Despite having considerably fewer neural network parameters to optimize, the policy optimized with the multihead version yielded a similar reward as the single-head version [see Fig. 8(b)]. This indicates that sharing the neural network backbone is permissible for each cost value function, and the network capacity is sufficient to model them. Because only the output layer size increases in proportion to the number of constraints, the suggested technique is highly scalable and effective for solving problems with numerous constraints.

In addition, the multihead version showed the least computation time among the three methods due to its parallelized cost value function training and cost advantage computation (see Table VI). The method’s strength in terms of computation time grows as the number of constraints increases because, unlike other approaches, multihead computation time does not increase linearly. The computation time for the cost value function training and cost advantage computation is not ignorable for all three methods, considering the relative ratio of time spent on other processes (e.g., simulation), and can be a bottleneck, especially for the single-head method, when the number of constraints increases. If we reduce the size of the cost value network, as done in the *single-head (reduced)* method, the computation time problem can be alleviated. However, insufficient network capacity deteriorates the cost value prediction accuracy, exhibiting high value prediction loss with high variance. As the number of constraints increases, this becomes greater, affecting the derivation of an appropriate policy gradient from the computed cost advantage value.

3) *Algorithm Hyperparameters*: We analyzed the sensitivity for two hyperparameter types: t , which determines the gradient profile of the logarithmic barrier functions, and α , which determines how much the feasible region will be enlarged for the adaptive constraint thresholding. The results are illustrated in Fig. 8(c). If the barrier function was set to have gradient penalties concentrated near the threshold (i.e., large t) or the feasible region was enlarged too much (i.e., large α), the barrier function applied small policy gradients to guide the policy toward the intended region, resulting in some constraints to be violated after the policy convergence (see Table II). If the barrier function was modified to be far from an indicator function and overall have a smooth gradient profile (i.e., small t), the policy was trained to satisfy constraints with large margins, resulting in a low task reward acquisition (in our case, command tracking reward). If the feasible region was enlarged very slightly (i.e., small α), it showed a small policy update step during the feasibility checking by the line search of the TRPO optimizer, especially at the initial training stage when the policy’s exploration ratio is high. This can make the training slow depending on the policy exploration method and the training environment.

Although the training characteristics change depending on the hyperparameters t and α , the trained results' variances were not very large in terms of constraint satisfaction, even though we modified hyperparameters on a large scale of ten. Furthermore, for all robots and various constraint designs used in our experiment, only a single set of hyperparameters, $t = 100$ and $\alpha = 0.02$, was used and resulted in reliable constraint satisfaction and superior reward acquisition. This shows that the hyperparameters are robust across different RL environments once the suitable value is selected.

VII. CONCLUSION

We proposed a learning framework for training neural network controllers for complex robotic systems consisting of both rewards and constraints. Suitable constraint types were suggested, where each can be configured in a way that best reflects the engineer's intent. An efficient policy optimization algorithm was then proposed, based on the previous works on constrained RL literature, to search for a policy that maximizes the reward while satisfying multiple constraints. Our learning framework was applied for training controllers for legged robot locomotion in challenging terrains, which has previously been done with considerable effort in reward engineering. Extensive simulation and real-world experiments with diverse robots, possessing different morphologies and physical properties, showed the generalizability and capability of using constraints for training performant controllers with significantly less reward engineering. Further analysis of *objective sensitivity* and *constraint satisfaction* confirms that, in comparison with employing only rewards, the proposed framework can make the engineering process of generating desired motions more straightforward and efficient. To the best of the authors' knowledge, our work is the first to show highly robust real-world robot performance for multiple legged robots using constrained RL and demonstrate the benefits of constraints from an engineering standpoint.

We strongly believe that our work suggests a new direction for training neural network controllers for robotic systems. We provide a new perspective and capability of leveraging constraints in the learning pipeline, which replaces the substantial and laborious reward engineering that was unavoidable in the reward-only approach. Promising directions for future works include developing novel constraint formulations with different kernel functions based on specific use cases. It will also be fascinating to see how the learning framework is applied to diverse tasks and robot platforms. Lastly, leveraging several advanced exploration methods for training complex motions, rather than the basic random sampling from the Gaussian distribution, is promising in terms of making the training more data-efficient.

APPENDIX

A. Comparison of Velocity Command Tracking Performance With the Reward-Only Framework

As shown in Fig. 6(a), our approach performed worse at velocity command tracking than the policy trained using the reward-only framework for the Raibo robot. This is due to the different

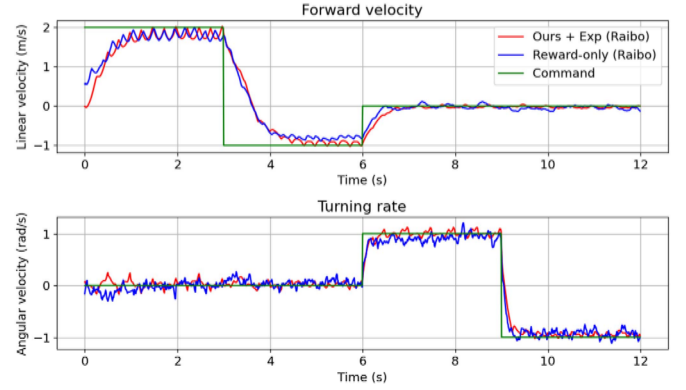


Fig. 9. Maximum velocity command tracking results.

kernel functions used for the command tracking reward. In our method, a quadratic function was used for simplicity, where the command tracking reward is defined as $r_c = -k_c(\|\text{cmd}_{v_{xy}} - V_{xy}\|^2 + (\text{cmd}_{w_z} - w_z)^2)$. In the reward-only framework, an exponential function was used following the previous work [8], [9], where the command tracking reward is defined as $r_c = k_c(\exp(-\|\text{cmd}_{v_{xy}} - V_{xy}\|^2) + \exp(-1.5(\text{cmd}_{w_z} - w_z)^2))$.

As given in Table VII, we can achieve the same command tracking performance with similar torque usage within our framework by applying the same exponential function for the command tracking reward and adjusting the reward coefficients accordingly ($k_c = 200$, $\hat{k}_\tau = 0.03$). When compared with a quadratic function, an exponential function has a steeper gradient near zero, resulting in more sensitive reward feedback for different command tracking errors. As a result, the robot can be trained to exhibit smaller command tracking errors. In addition, similar maximum speeds can be achieved when given the maximum forward velocity ($-1 \text{ m/s} \sim 2 \text{ m/s}$) and turning rate ($-1 \text{ rad/s} \sim 1 \text{ rad/s}$) commands (see Fig. 9).

To verify the robot generalization ability with different reward designs, we retrained the controller for the ANYmal C robot with the same reward coefficients used for the Raibo robot but with an exponential reward kernel (see Table VII). Similar results with Fig. 6(a) were obtained, indicating the maintenance of the robot generalization capability. This is because the critical component for achieving robot generalizability is the incorporation of constraints rather than specific reward designs. The proposed method offers strong robot generalizability as long as constraints are introduced and the effect is regardless of reward designs.

B. Extra Demonstration of Constraint's Engineering Advantages

The engineering benefits of constraints over rewards are demonstrated further with a simple toy example and a thorough comparison of the entire engineering scenario. Starting from a simple RL setup with just two rewards, the goal is to train a neural network controller that exhibits a trotting gait pattern. The defined two rewards are designed for the robot to move forward with a velocity higher than 1 m/s with minimal torque usage. The toy example setting is taken from the RaiSim [56]

TABLE VIII
ENGINEERING SCENARIO COMPARISON

	Iteration	Parameters	Evaluation				Comment
			Task	Style / Safety			
			Velocity reward	Body contact	Contact velocity	Gait difference	
Initial configuration		$k_v = 0.3$, $k_\tau = 4e - 5$	0.94	0.32	1.10	0.50	- Too much body contact - Too much foot slippage - Not following the desired gait
Reward-only	1	$k_{bc} = 1$	0.95	0.21	0.96	0.52	(A) Set body contact reward (R) Still too much body contact
	2	$k_{bc} = 1e + 2$	0.94	0.02	2.10	0.50	(A) Set a higher body contact reward coefficient (R) Still frequent body contact
	3	$k_{bc} = 1e + 4$	0.96	0.00	1.23	0.49	(A) Set a higher body contact reward coefficient (R) Nearly zero body contact (R) Move on to reducing foot slippage
	4	$k_{bc} = 1e + 4$, $k_{cv} = 1$	0.95	0.00	1.28	0.50	(A) Set slippage reward (R) Still too much foot slippage
	5	$k_{bc} = 1e + 4$, $k_{cv} = 1e + 2$	0.96	0.00	0.17	0.50	(A) Set a higher slippage reward coefficient (R) Still frequent foot slippage
	6	$k_{bc} = 1e + 4$, $k_{cv} = 1e + 4$	0.78	0.00	0.05	0.50	(A) Set a higher slippage reward coefficient (R) Acceptable foot slippage. (R) However, degradation in task performance .
	7	$k_{bc} = 1e + 4$, $k_{cv} = 1e + 3$	0.92	0.00	0.09	0.50	(A) Set a smaller slippage reward coefficient (R) Acceptable slippage (R) Move on to tracking the desired gait
	8	$k_{bc} = 1e + 4$, $k_{cv} = 1e + 3$, $k_{gp} = 1e + 2$	0.91	0.00	0.12	0.49	(A) Set gait reward (R) Not tracking the desired gait
	9	$k_{bc} = 1e + 4$, $k_{cv} = 1e + 3$, $k_{gp} = 1e + 4$	0.87	0.00	0.12	0.03	(A) Set a higher gait reward coefficient (R) Acceptable gait tracking performance (R) However, slight increase in foot slippage .
Reward + Constraints (Ours)	1	$D_{bc} = 0.025$	0.96	0.00	1.44	0.51	(A) Set body contact constraint with near-zero threshold (R) Nearly zero body contact (R) Move on to reducing foot slippage
	2	$D_{bc} = 0.025$, $D_{cv} = 0.1$	0.95	0.00	0.05	0.50	(A) Set slippage constraint with 0.1 m/s threshold (R) Acceptable slippage (R) Move on to tracking the desired gait
	3	$D_{bc} = 0.025$, $D_{cv} = 0.1$, $D_{gp} = 0.025$	0.92	0.00	0.09	0.024	(A) Set gait pattern constraint with near-zero threshold (R) Acceptable gait tracking performance

* “(A)” indicates the engineer’s action. “(R)” indicates the engineer’s response after reviewing the trained results for the action taken in “(A).”

Blue-colored words indicate the engineer’s intended results, Red-colored words indicate the engineer’s unintended results, Green-colored words indicate the representative results

tutorial and the reward coefficients are also set the same. The complete engineering scenario is provided in Table VIII.

When the robot is trained with just these two rewards, it exhibits the following three problems:

- 1) too much undesirable contact (e.g., self-collision, shank contact);
- 2) too much foot slippage; and
- 3) not exhibiting the desired gait pattern (i.e., trot).

When constraints can be used during the training process, the above three problems can be solved easily by defining constraints appropriately. Based on the characteristics of each constraint type that we explained above, we defined three constraints: body contact constraint (probabilistic constraint, threshold = 0.025), contact velocity constraint (average constraint, threshold = 0.1 m/s), and gait pattern constraint (probabilistic constraint, threshold = 0.025). By adding just these three constraints, we successfully trained a trotting controller with the above three problems solved. The video is available online (<https://youtu.be/M-H1z2QRLeQ>), which shows how the policy evolves by introducing constraints one at a time. Similar controllers can be obtained by defining the cost functions as

rewards and assigning appropriate reward coefficients to each of them. However, because reward coefficients can only model the relative relationships between each term and the trained result for a specific reward coefficient set is unpredictable, it takes several trials and errors to find the best coefficient set that can lead to the desired result. Furthermore, these coefficients must be carefully tuned so that they do not interfere with the main task performance (i.e., velocity reward). We believe that the corresponding experiment showcases how using constraints with rewards can make the entire engineering process of representing the engineer’s intent more straightforward and effective.

REFERENCES

- [1] J. Hwangbo et al., “Learning agile and dynamic motor skills for legged robots,” *Sci. Robot.*, vol. 4, no. 26, 2019, Art. no. eaau5872.
- [2] Y. Pan et al., “Agile autonomous driving using end-to-end deep imitation learning,” in *Proc. Robot.: Sci. Syst.*, Pittsburgh, Pennsylvania, 2018.
- [3] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proc. Robot.: Sci. Syst.*, Daegu, Republic of Korea, 2023.

- [4] X. B. Peng, P. Abbeel, S. Levine, and M. V. d. Panne, "Deepmimic: Example-guided deep reinforcement learning of physics-based character skills," *ACM Trans. Graph.*, vol. 37, no. 4, pp. 1–14, 2018.
- [5] J. Hwangbo, I. Sa, R. Siegwart, and M. Hutter, "Control of a quadrotor with reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 2, no. 4, pp. 2096–2103, Oct. 2017.
- [6] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Sci. Robot.*, vol. 5, no. 47, 2020, Art. no. eabc5986.
- [7] O. M. Andrychowicz et al., "Learning dexterous in-hand manipulation," *Int. J. Robot. Res.*, vol. 39, no. 1, pp. 3–20, 2020.
- [8] G. Ji, J. Mun, H. Kim, and J. Hwangbo, "Concurrent training of a control policy and a state estimator for dynamic and robust legged locomotion," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 4630–4637, Apr. 2022.
- [9] S. Choi et al., "Learning quadrupedal locomotion on deformable terrain," *Sci. Robot.*, vol. 8, no. 74, 2023, Art. no. eade2256.
- [10] S. Gangapurwala, A. Mitchell, and I. Havoutis, "Guided constrained policy optimization for dynamic quadrupedal robot locomotion," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 3642–3649, Apr. 2020.
- [11] J. Lee, L. Schroth, V. Klemm, M. Bjelonic, A. Reske, and M. Hutter, "Evaluation of constrained reinforcement learning algorithms for legged locomotion," 2023, *arXiv:2309.15430*.
- [12] X. B. Peng, Z. Ma, P. Abbeel, S. Levine, and A. Kanazawa, "AMP: Adversarial motion priors for stylized physics-based character control," *ACM Trans. Graph.*, vol. 40, no. 4, pp. 1–20, 2021.
- [13] A. Nagabandi, K. Konolige, S. Levine, and V. Kumar, "Deep dynamics models for learning dexterous manipulation," in *Proc. Conf. Robot Learn.*, 2020, pp. 1101–1112.
- [14] Y. Kim, C. Kim, and J. Hwangbo, "Learning forward dynamics model and informed trajectory sampler for safe quadruped navigation," in *Proc. Robot.: Sci. Syst.*, New York City, NY, USA, 2022.
- [15] N. Heess et al., "Emergence of locomotion behaviours in rich environments," 2017, *arXiv:1707.02286*.
- [16] Z. Fu, X. Cheng, and D. Pathak, "Deep whole-body control: Learning a unified policy for manipulation and locomotion," in *Proc. Conf. Robot Learn.*, 2022, pp. 138–149.
- [17] C. D. Bellicoso, F. Jenelten, C. Gehring, and M. Hutter, "Dynamic locomotion through online nonlinear motion optimization for quadrupedal robots," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 2261–2268, Jul. 2018.
- [18] J. D. Carlo, P. M. Wensing, B. Katz, G. Bleth, and S. Kim, "Dynamic locomotion in the MIT Cheetah 3 through convex model-predictive control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 1–9.
- [19] F. Jenelten, R. Grandia, F. Farshidian, and M. Hutter, "TAMOLS: Terrain-aware motion optimization for legged systems," *IEEE Trans. Robot.*, vol. 38, no. 6, pp. 3395–3413, Dec. 2022.
- [20] R. Grandia, F. Jenelten, S. Yang, F. Farshidian, and M. Hutter, "Perceptive locomotion through nonlinear model-predictive control," *IEEE Trans. Robot.*, vol. 39, no. 5, pp. 3402–3421, Oct. 2023.
- [21] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Sci. Robot.*, vol. 7, no. 62, 2022, Art. no. eabk2822.
- [22] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proc. Conf. Robot Learn.*, 2022, pp. 91–100.
- [23] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018, pp. 3803–3810.
- [24] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid motor adaptation for legged robots," in *Proc. Robot.: Sci. Syst.*, Virtual, 2021.
- [25] A. Agarwal, A. Kumar, J. Malik, and D. Pathak, "Legged locomotion in challenging terrains using egocentric vision," in *Proc. Conf. Robot Learn.*, 2023, pp. 403–415.
- [26] G. B. Margolis, G. Yang, K. Paigwar, T. Chen, and P. Agrawal, "Rapid locomotion via reinforcement learning," in *Proc. Robot.: Sci. Syst.*, New York City, NY, USA, 2022.
- [27] Y. Shao, Y. Jin, X. Liu, W. He, H. Wang, and W. Yang, "Learning free gait transition for quadruped robots via phase-guided controller," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 1230–1237, Apr. 2022.
- [28] Y. Kim, B. Son, and D. Lee, "Learning multiple gaits of quadruped robot using hierarchical reinforcement learning," 2021, *arXiv:2112.04741*.
- [29] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, "Real-world humanoid locomotion with reinforcement learning," *Sci. Robot.*, vol. 9, no. 89, p. eadi9579, 2024.
- [30] D. Hoeller, N. Rudin, D. Sako, and M. Hutter, "ANYmal Parkour: Learning agile navigation for quadrupedal robots," *Sci. Robot.*, vol. 9, no. 88, p. eadi7566, 2024.
- [31] X. Cheng, A. Kumar, and D. Pathak, "Legs as manipulator: Pushing quadrupedal agility beyond locomotion," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 5106–5112.
- [32] A. Escontrela et al., "Adversarial motion priors make good substitutes for complex reward functions," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 25–32.
- [33] J. Wu, G. Xin, C. Qi, and Y. Xue, "Learning robust and agile legged locomotion using adversarial motion priors," *IEEE Robot. Autom. Lett.*, vol. 8, no. 8, pp. 4975–4982, Aug. 2023.
- [34] Y. Fuchioka, Z. Xie, and M. V. d. Panne, "OPT-mimic: Imitation of optimized trajectories for dynamic quadruped behaviors," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 5092–5098.
- [35] G. Feng et al., "GenLoco: Generalized locomotion controllers for quadrupedal robots," in *Proc. Conf. Robot Learn.*, 2023, pp. 1893–1903.
- [36] E. Altman, *Constrained Markov Decision Processes*. London, U.K.: Routledge, 2021.
- [37] J. Garcia and F. Fernández, "A comprehensive survey on safe reinforcement learning," *J. Mach. Learn. Res.*, vol. 16, no. 1, pp. 1437–1480, 2015.
- [38] T.-Y. Yang, T. Zhang, L. Luu, S. Ha, J. Tan, and W. Yu, "Safe reinforcement learning for legged locomotion," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 2454–2461.
- [39] H. Bharadhwaj, A. Kumar, N. Rhinehart, S. Levine, F. Shkurti, and A. Garg, "Conservative safety critics for exploration," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [40] Y. Chow, M. Ghavamzadeh, L. Janson, and M. Pavone, "Risk-constrained reinforcement learning with percentile risk criteria," *J. Mach. Learn. Res.*, vol. 18, no. 1, pp. 6070–6120, 2017.
- [41] C. Tessler, D. J. Mankowitz, and S. Mannor, "Reward constrained policy optimization," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [42] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 22–31.
- [43] Y. Liu, J. Ding, and X. Liu, "IPO: Interior-point policy optimization under constraints," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 04, 2020, pp. 4940–4947.
- [44] L. Zhang et al., "Penalized proximal policy optimization for safe reinforcement learning," 2022, *arXiv:2205.11814*.
- [45] T. Xu, Y. Liang, and G. Lan, "CRPO: A new approach for safe reinforcement learning with convergence guarantee," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 11480–11491.
- [46] T.-Y. Yang, J. Rosca, K. Narasimhan, and P. J. Ramadge, "Projection-based constrained policy optimization," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [47] A. Ray, J. Achiam, and D. Amodei, "Benchmarking safe exploration in deep reinforcement learning," 2019, *arXiv:1910.01708*.
- [48] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1889–1897.
- [49] J. Schulman, P. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [50] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2015, *arXiv:1506.02438*.
- [51] Z. Xie, H. Y. Ling, N. H. Kim, and M. v. d. Panne, "ALLSTEPS: Curriculum-driven learning of stepping stone skills," *Comput. Graph. Forum*, vol. 39, no. 8, pp. 213–224, 2020.
- [52] W. Yu, G. Turk, and C. K. Liu, "Learning symmetric and low-energy locomotion," *ACM Trans. Graph.*, vol. 37, no. 4, pp. 1–12, 2018.
- [53] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, *arXiv:1312.6114*.
- [54] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *Proc. Annu. Conf. Neural Inf. Process. Syst. Workshop Deep Learn.*, 2014.
- [55] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inform. Process. Syst.*, 2019, vol. 32.
- [56] J. Hwangbo, J. Lee, and M. Hutter, "Per-contact iteration method for solving contact dynamics," *IEEE Robot. Autom. Lett.*, vol. 3, no. 2, pp. 895–902, Apr. 2018.
- [57] B. Katz, J. D. Carlo, and S. Kim, "Mini Cheetah: A platform for pushing the limits of dynamic quadruped control," in *Proc. Int. Conf. Robot. Autom.*, 2019, pp. 6295–6301.
- [58] Y.-H. Shin et al., "Design of KAIST HOUND, a quadruped robot platform for fast and efficient locomotion with mixed-integer nonlinear optimization of a gear train," in *Proc. Int. Conf. Robot. Autom.*, 2022, pp. 6614–6620.
- [59] M. Hutter et al., "ANYmal-a highly mobile and dynamic quadrupedal robot," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2016, pp. 38–44.



Yunho Kim received the B.Sc. degree in mechanical engineering from Seoul National University, Seoul, Republic of Korea, in 2022, and is currently working toward the master's degree in mechanical engineering from the Korea Advanced Institute of Science and Technology, Daejeon, South Korea.

His research interests include robotics and artificial intelligence lab, under the supervision of Prof. J. Hwangbo.



Gwanghyeon Ji received the B.Sc. degree in mechanical engineering from the Pohang University of Science and Technology, Pohang, Republic of Korea, in 2017, and the M.Sc. degree in mechanical engineering in 2023 from the Korea Advanced Institute of Science and Technology, Daejeon, South Korea, where he is currently working toward the Ph.D. degree in mechanical engineering.

His main research interests include and Artificial Intelligence Lab, under the supervision of Prof. J. Hwangbo.



Hyunsik Oh (Graduate Student Member, IEEE) received the B.Sc. degree in mechanical engineering from Hanyang University, Seoul, Republic of Korea, in 2023, and is currently working toward the master's degree in mechanical engineering from the Korea Advanced Institute of Science and Technology, Daejeon, South Korea.

His research interests include robotics and artificial intelligence lab, under the supervision of Prof. J. Hwangbo.



Moonkyu Jung received the B.Sc. degree in mechanical engineering in 2023 from the Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea, where he is currently working toward the master's degree in mechanical engineering.

His research interests include Robotics and Artificial Intelligence Lab, under the supervision of Prof. J. Hwangbo.



Jeonghyun Lee (Graduate Student Member, IEEE) received the B.Sc. degree in mechanical engineering in 2022 from the Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea, where he is currently working toward the master's degree in the graduate school of AI from the Korea Advanced Institute of Science and Technology (KAIST) in Daejeon, South Korea.

His research interests include Robotics and Artificial Intelligence Lab, under the supervision of Prof. J. Hwangbo.



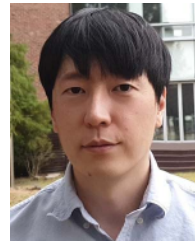
Donghoon Youm (Graduate Student Member, IEEE) received the B.Eng. and M.Eng. degrees in mechanical engineering in 2020 and 2022, respectively, from the Korea Advanced Institute of Science and Technology (KAIST), where he is currently working toward the Ph.D. degree in mechanical engineering.

His research interests include Robotics and Artificial Intelligence Lab, under the supervision of Prof. J. Hwangbo.



Jinhyeok Choi is currently working toward the under graduation degree in mechanical engineering from the Korea Advanced Institute of Science and Technology, Daejeon, South Korea, under the supervision of Prof. J. Hwangbo.

His research interests include legged robot actuators and locomotion.



Jemin Hwangbo (Member, IEEE) received the B.S. degree from the University of Toronto, Toronto, ON, Canada, in 2006, and the M.S. and Ph.D. degrees from ETH Zurich, Zurich, Switzerland, in 2013 and 2019, respectively.

He is currently an Assistant Professor with the Department of Mechanical Engineering, Korea Advanced Institute of Science and Technology, Daejeon, South Korea, and concurrently holds the position of Director with Robotics and Artificial Intelligence Lab. His group's research focuses on primarily centered around legged robotics, encompassing areas, such as design, vision, control, and navigation.

centered around legged robotics, encompassing areas, such as design, vision, control, and navigation.